拓展：PDS日活拆解与关联知识网络

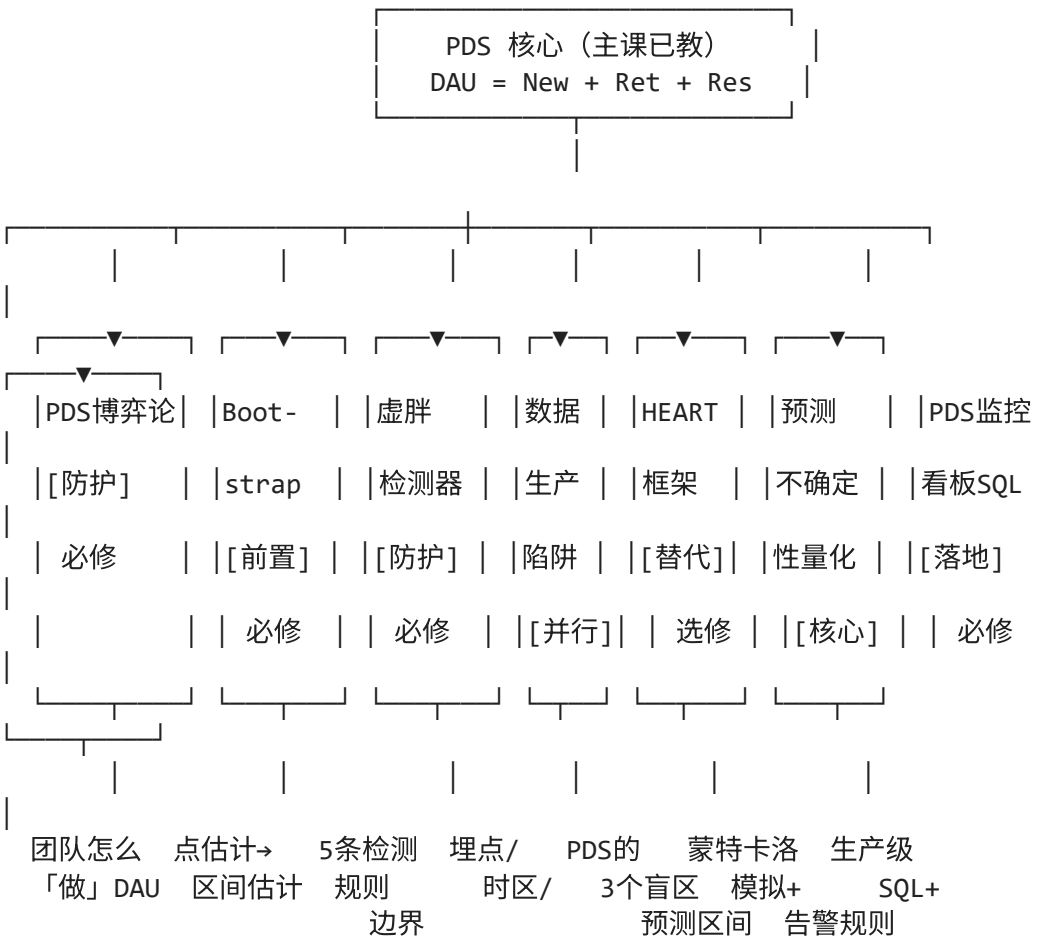
> **本拓展新增 7 个课程未覆盖的深度概念**，其中 4 个来自全课程永久空白区。 阅读时间：约 2-2.5 小时 | [必修]4个 | [选修]2个 | [预览]1个

📖 阅读指引

本拓展与主课的关系：主课教你 PDS 的「是什么」和「怎么用」——你已经知道 DAU = New + Retained + Resurrected，知道如何判断增长模式和虚胖 DAU。本拓展做三件事：

1. **往前挖**：你看到的 PDS 数字有多可靠？占比差了 5pp 是「真变化」还是「随机波动」？（Bootstrap 置信区间）
2. **往深挖**：PDS 预测的 103 万 vs 95 万——哪个更可信？不确定性从哪来？（蒙特卡洛预测区间）
3. **往广挖**：当「DAU」变成 KPI 后团队会怎么博弈？（博弈论）数据生产环节有哪些污染？（工程陷阱）PDS 的盲区是什么？（HEART 互补）能否自动化？（检测器）

知识网络预览：



概念关系速览：

#	概念	关系类型	深度类型	必修/选修	来源
1	PDS博弈论——DAU为KPI时的团队博弈	[防护]	博弈+案例	必修	永久空白区
2	Bootstrap置信区间——占比变化是信号还是噪声	[前置]	数学+代码	必修	永久空白区
3	虚胖DAU自动检测器——从人工到自动化	[防护]	代码	必修	永久空白区
4	PDS数据生产陷阱——埋点/时区/边界	[并行]	工程+代码	选修	工程实践
5	PDS监控看板SQL——从概念到生产	[落地]	工程+代码	必修	工程实践
6	HEART框架——Google UX度量如何弥补PDS盲区	[替代]	案例	选修	永久空白区
7	DAU预测的不确定性量化——蒙特卡洛模拟	[核心深化]	数学+代码	必修	数学深化

你将在本拓展中构建的「沙盘」：一家短视频社交产品，DAU 半年涨了 50%，但增长质量存疑。你将扮演高级数据分析师，用 7 个深度概念和一张完善的监控看板，向 CEO 揭示增长真相。

第一部分：沙盘设定

🏢 沙盘：公司画像 + 产品 + 商业模式

公司：闪念科技（FlashMind），创立 3 年，C 轮融资，750 人规模。

产品：「闪念」——15 秒短视频 + 直播社交 App。核心功能：信息流短视频浏览、直播互动、私信聊天。DAU 约 120 万（Android 60%, iOS 30%, Web 10%）。

商业模式：信息流广告（65% 收入）+ 直播打赏分成（30%）+ 会员订阅（5%）。日投放预算约 12 万元（信息流 8 万 + 搜索 3 万 + 社媒 1 万）。

行业背景：短视频赛道进入存量竞争期——头部产品 DAU 过亿但增速放缓到个位数。闪念作为第三梯队产品，面临「不投就跌、投了才涨」的增长困境。

🎯 当前挑战 + 你的角色

当前挑战：过去 6 个月，DAU 从 80 万涨到 120 万（+50%），管理层在全员会上庆祝。但运营 VP 私下跟你说：「我直觉不对——留存率的数据好像在下滑，但投放团队说'DAU 在涨说明策略有效'。你帮我看看增长到底健不健康。」

你的角色：闪念科技高级数据分析师（入职 8 个月）。你负责 DAU 健康度监控体系，直接汇报给运营 VP。你需要在周五的月度经营分析会上，向 CEO 和 VP 们展示 DAU 增长的「真相」——不只是「涨了 50%」这个数字，而是背后的结构变化。

利益相关方：

- **CEO 李总**：关注规模——「明年要到 300 万 DAU，给我看增长路径」
- **运营 VP 王姐**：关注留存——「DAU 涨了但用户质量好像在恶化」
- **投放总监 张哥**：关注 ROI——「给我更多预算，我能把 DAU 顶到 200 万」
- **产品总监 陈姐**：关注体验——「别光看 DAU，用户真的喜欢我们吗？」

📊 数据资产（5 张核心表 + 字段）

表名	用途	关键字段	更新频率
`flashmind.user_active_log`	用户日活明细	user_id, active_date, platform, app_version, session_cnt, total_duration_sec	每日 T+1
`flashmind.user_lifecycle`	用户生命周期状态	user_id, register_date, first_active_date, last_active_date, lifecycle_status(new/retained/resurrected/dor	每日 T+1
`flashmind.channel_performance`	投放渠道效果	channel, date, cost, new_users, impressions, clicks, ctr, cpm	每日 T+1
`flashmind.push_log`	Push 推送日志	user_id, push_date, push_type, is_clicked, is_converted	每日 T+1
`flashmind.retention_matrix`	留存矩阵	cohort_date, day_n, cohort_size, retained_users, retention_rate	每日 T+1

第二部分：入口自诊（只测主课内容）

🩺 自诊题（5 题，5 分钟）

> 这些题只测主课教的 PDS 基础——如果你答不上来，先回主课复习再看拓展。

T1：DAU = 100 万。New 12 万，Retained 65 万，Resurrected 23 万。判断增长模式，并说明 Retained 占比是否健康。

T2：某产品 DAU 从 200 万涨到 250 万，但 Retained 从 150 万跌到 140 万。New 从 30 万涨到 80 万。这是真增长还是虚胖？用数据说明。

T3：写出 PDS 贡献度公式。如果 DAU 总变化 +20 万，New 变化 +25 万，Retained 变化 -5 万——各部分的贡献度是多少？Retained 贡献度 < 0 意味着什么？

T4：Retained 在降。短期留存（1-7 天）稳定，中期留存（7-30 天）明显下降——最可能的原因是什么？

T5：为什么说「Retained 绝对值是产品健康度最诚实的信号」？Retained 和新/Resurrected 的根本区别是什么？

📋 自诊反馈

T1 答案：Retained 占比 65% > 60% → 留存驱动型，健康。New 仅占 12% → 增长不依赖投放。

> ❓ 常见偏差 1：只看了占比没看趋势 → Retained 65% 但如果在连续下降 → 也需要关注 ❓ 常见偏差 2：Retained 65% 但在 12% 的 New 加持下 DAU 可能增长缓慢 → 留存驱动型如果规模不涨也要关注拉新不足 🔧 补救：回顾主课 cell-1 的「三种模式判断标准」

T2 答案：典型的虚胖 DAU。DAU 涨了 50 万但 Retained 跌了 10 万（-6.7%），增长 100% 靠 New（+167%）。New 贡献度 = 50/50 = 100%，Retained 贡献度 = -10/50 = -20%。

> ❓ 常见偏差 1：只看 DAU 涨了 25% 就觉得好 → 忽略了内部「用户置换」问题 ❓ 常见偏差 2：只看贡献度不看绝对值 → 如果 Retained 贡献度 +10% 但绝对值从 1 万→1.1 万（基数太小），也不算健康 🔧 补救：回顾主课 cell-8 的「四维 PDS 诊断」

T3 答案：贡献度 = 该部分变化量 ÷ DAU 总变化量 × 100%。New 贡献度 = 25/20 × 100% = 125%（New 增长超过了 DAU 总增长，说明其他部分在拖后腿）。Retained 贡献度 = -5/20 × 100% = -25%。Retained 贡献度 < 0 → 核心用户资产在萎缩，抵消了拉新的效果。

> ❓ 常见偏差 1：贡献度直接相加应该等于 100%（New 125% + Ret -25% = 100%）→ 如果不等于 100% 说明计算有误 ❓ 常见偏差 2：忘记看 Resurrected 的贡献度 → 本场景中 Resurrected 没变化所以贡献 0%，但也可能是 Resurrected 在暗中变化 🔧 补救：回顾主课 cell-10 的「贡献度解读规则」

T4 答案：中期留存（7-30 天）下降 → 产品的「习惯养成」能力出问题了。用户试用了 1 周但没有形成使用习惯。可能原因：内容更新不够频繁、缺乏个性化推荐、缺少社交连接、push 节奏不当。

> ❓ 常见偏差 1：直接跳到「竞品抢用户」→ 应该先排查内部原因（产品改版/内容策略/推荐算法）再查外部 ❓ 常见偏差 2：认为「短期留存稳定就没事」→ 短期留存是「用户不讨厌你」，中期留存才是「用户需要你」 🔧 补救：回顾主课 cell-1 的「Retained 深层拆解」和 cell-14 的告警规则

T5 答案：Retained 是「免费」的增长——用户昨天用了、今天主动回来，不需要投放或 push 推动。New 要花钱（投放），Resurrected 要催活（push/短信）。所以 Retained 绝对值的趋势 = 「产品自然吸引力」的变化趋势。Retained 跟 New/Resurrected 的根本区别：前者是用户主动选择，后两者需要外部刺激。

> ❓ 常见偏差 1：认为「Retained 也受外部因素影响」→ 对的，但影响的是幅度而非是否回来 ❓ 常见偏差 2：认为「Resurrected 也是主动回来的」→ 可能有部分是主动（想起来就打开），但大多数是被 push/活动「催」回来的，区别在于「催回来的」次日大概率又走 🔧 补救：回顾主课 cell-0 的「最重要认知」和 cell-1 的「三类用户的行为模式」

> ⏭️ 如果 5 题都答对 → 进入拓展。如果有 2 题以上答错 → 先回主课复习对应 cell 再看拓展。

第三部分：概念网络展开

🔴 概念 1：PDS 博弈论——当「DAU」成为 KPI

> 关系类型：[防护] | 深度类型：博弈 + 案例 | [必修] | 来源：永久空白区·指标腐烂生命周期

🔴 认知冲突

场景：你发现了一个奇怪的数据现象——「闪念」的 New 用户每天都在涨，但次留率从 35% 暴跌到 12%。更奇怪的是，新增用户的平均 session 时长只有 3 秒——几乎就是「打开→关闭」。你问投放团队怎么回事，对方回答：「素材优化了，CTR 提高了 40%！」

问题：如果投放团队的 KPI 是「日均新增用户数」，他们有什么动机去优化 CTR 而非次留？他们「做对了」KPI 还是「做好了」产品？

停下来想想（试着回答再往下看）：如果明天 CEO 把 DAU 设为你个人的 KPI，给你 100 万月预算，你会怎么做？你会选择 (a) 砸钱买量 (b) 改进产品留存 (c) 加 push 频率？每种选择的后果是什么？

概念引入

① **一句话白话**：当你把「DAU 总量」设为 KPI 的时候，你实际上在告诉所有人——「我不管你用什么方法，让这个数字变大就行。」然后人们会找到最省力的方法让它变大，而这些方法往往跟「做好产品」毫无关系。

② **核心逻辑 + 沙盘案例**：

Goodhart's Law（古德哈特定律）的 DAU 版本：**当一个指标成为目标，它就不再是一个好指标。** 因为人们会开始「优化指标本身」而非「优化指标背后的用户价值」。

在闪念的沙盘中，有三条完全不同的「做 DAU」路径，每条路径对应不同的博弈模式：

路径	怎么做	KPI 数字变化	用户价值变化	博弈者	可持续性
A: 砸钱买量	加投放预算，信息流素材从「精准卖点」改成「标题党」	New ↑↑, DAU ↑	↓（来的人不匹配→体验差→走）	投放团队	低，边际成本递增
B: 强 push 催活	push 频率从 2 次/周→3 次/天，用户不胜其烦	Resurrected ↑↑, DAU ↑	↓（推送给脱敏→卸载率↑）	运营团队	低，脱敏递减
C: 改进产品	优化推荐算法，提升内容供给质量	Retained ↑(慢), DAU ↑(慢)	↑（用户体验改善→主动回来）	产品+算法	高，复利效应

博弈的本质：A 和 B 的「KPI 见效速度」远快于 C——投放预算加一倍，明天 New 就翻倍；push 频率加一倍，明天 Resurrected 就翻倍。但 C 需要改产品、上线、验证——至少 2-4 周才看到 Retained 变化。在一个「月度考核 DAU」的体系下，没有人会选择 C——因为 C 的见效速度会让你在月度考核时垫底。

这就是指标博弈的悲剧：**考核短期 DAU → 鼓励砸钱买量 → 留存恶化 → 更需要砸钱才能维持 DAU → 对投放产生「药物依赖」→ 留存继续恶化 → ……→ 崩盘。**

③ 与 PDS 的关系：[防护]——你知道 PDS 能「看出来」虚胖。但更往前一步：你能不能设计一个「让博弈无法得逞」的指标体系？把考核指标从「DAU 总量」变成「DAU 总量 × 留存率加权」？把投放团队的 KPI 从「新增量」改成「新增量 × 次留率」？

🏢 沙盘用武之地

在闪念科技，你发现投放总监张哥的 KPI 是「月均新增用户数」。他的团队有强烈的动机：

- 用「标题党」素材提升 CTR（反正新增量考核的是下载并打开 App，标题党最容易诱导下载）
- 不关心次留（因为不是他的 KPI）
- 反对你提出的「渠道质量门槛」建议（因为会减少他的新增量）

你需要在下周的 KPI 修订会议上提出：把张哥的 KPI 从「月均新增用户数」改成「月均新增用户数 × 次留率 × 14 日留存率」。这个改动会让他的激励从「多多益善」变成「优质多量」——因为低质渠道的次留率会把他的 KPI 拉下来。

博弈论教你的是：不要试图改变人性——改变规则。

✍️ 练习：识别你身边的数据博弈

回想一个你经历过或听说过的 KPI 被博弈的案例（可以是工作中的，也可以是公开报道的——比如某平台的「GMV 注水」、某 App 的「评论刷量」）。回答：

1. 被博弈的指标是什么？
2. 博弈者用了什么方法？
3. 如果让你重新设计这个指标，你会改成什么？

思考模板（以闪念为例）：

- 被博弈的指标：日均 New 用户数
- 博弈方法：用标题党素材拉高 CTR→下载量→低质用户涌入→次留暴跌
- 重新设计：日均有价值新增用户数（Valuable New）= count(user_id) WHERE first_day_session > 60sec AND day7_retained = 1

📋 反馈

这道练习没有「标准答案」——因为每个组织的博弈模式都不同。但有一个通用的检验框架：

博弈免疫指标的三条检验：

1. **能否被「花钱」直接推高？** → 能 → 危险（投放、红包、补贴都可以直接推高）
2. **能否被「机器」批量制造？** → 能 → 危险（脚本刷量、批量注册）
3. **指标上升 = 用户价值上升吗？** → 不是 → 危险（标题党推送带来新增，但用户价值在降）

> ❓ 常见偏差 1：认为「加上反作弊就能防止博弈」→ 反作弊只能防「假用户」，防不了「真用户但没价值的行为」（标题党来的真用户也是真用户，但他们不会留存） ❓ 常见偏差 2：认为「公司内部不会博弈，大家都在做好产品」→ 博弈不是「恶意」，是「激励导向的自然结果」——当你的季度奖金跟 Numbers 挂钩时，每个人都会下意识地优化 Numbers 🔧 补救路径：推荐阅读 Goodhart's Law 的原始论文（Goodhart, 1975），以及《Measuring the User Experience》中关于「指标博弈」的章节

🔴 概念 2：Bootstrap 置信区间——PDS 占比变化是「信号」还是「噪声」？

> 关系类型：[前置] | 深度类型：数学 + 代码 | [必修] | 来源：永久空白区·统计方法

🔴 认知冲突

场景：你看闪念上周的 PDS 报告——Retained 占比从 58% 降到了 54%（-4pp）。运营 VP 王姐看到这个数字很紧张：「Retained 在降！要不要启动留存专项？」

但你再仔细看看：上周日均 DAU 只有 7 个数据点（周一到周日），每天 DAU 约 120 万。7 个观测值算出的「54%」——这个数字有多可靠？如果下周自然波动回到 57%，你会不会白启动了专项？

问题：主课教你算 PDS 占比（如 Retained/DAU = 54%）——但它是一个「点估计」——单一的数字。你能不能说「我有 95% 的信心，真实的 Retained 占比在 51% 到 57% 之间」？如果能，这个区间是怎么算出来的？

停下来想想（试着回答再往下看）：如果你只有 7 天的 PDS 数据，你怎么判断 58%→54% 是「真下降」还是「正常波动」？

概念引入

① **一句话白话**：Bootstrap 就是「用自己的数据再造几千份假数据，看看你算出来的数字能波动多大」。波动大 = 你那 4pp 的变化可能只是随机运气；波动小 = 你确实看到了真实的变化。

② 核心逻辑 + 沙盘案例：

主课教你：Retained 占比 = Retained / DAU × 100%。但 7 天的数据只有 7 个占比值。从 7 个值中算出一个均值（54%）——这是点估计。问题：如果你换一周（不同 7 天），这个均值会变成 56% 还是 52%？我们不知道——因为一周的数据只是从「无限可能的数据集」中抽取的一个样本。

Bootstrap 的核心思想（Efron, 1979）：

```
Step 1: 你有 7 天数据 → 每天的 Retained 占比 = [58, 56, 59, 53, 52, 50, 49]（%）
Step 2: 从这 7 个值中有放回地随机抽 7 个 → 可能抽到 [56,56,53,50,59,52,58]
Step 3: 算这组「伪样本」的均值 → 54.8%
Step 4: 重复 Step 2-3 共 10,000 次 → 得到 10,000 个「伪均值」
Step 5: 这 10,000 个值的第 2.5 百分位和第 97.5 百分位 → 95% Bootstrap 置信区间
```

数学本质：Bootstrap 把「样本的经验分布」当作「总体的最佳估计」，然后从这个经验分布中反复抽样来模拟「如果我们重做这个实验，结果会怎么变化」。它不需要假设数据服从正态分布——这是它比传统 t 区间最大的优势。

对闪念的意义：如果 Retained 占比的 95% CI 是 [48%, 60%]——区间宽度 12pp，远大于你看到的 4pp 变化 → 你没有证据说 Retained 真的变了。如果 95% CI 是 [57.5%, 58.5%]——宽度只有 1pp → 4pp 的变化很大概率是真的。

③ 与 PDS 的关系：[前置]——主课教了 PDS 分解和贡献度的「点估计」，但没教如何判断这些数字是「信号」还是「噪声」。Bootstrap 给了你一个量化工具：任何 PDS 数字的变化都自带一个置信区间，在这个区间内的变化可能是噪声，超出区间的才值得深入排查。

🏢 沙盘用武之地

在闪念，你需要判断上周 Retained 占比降了 4pp 是否值得告警。你用 Bootstrap 对最近 30 天的 Retained 占比数据做重抽样，算出前 15 天的 95% CI 是 [56%, 60%]，后 15 天的 95% CI 是 [51%, 57%]。两个区间有重叠（[56%,57%] 区域），但后 15 天的区间明显下移。结论：「有证据表明 Retained 占比在降，但幅度尚不确定（可能在 -1pp 到 -5pp 之间）。建议再观察一周，同时准备留存专项的预案。」

如果只用点估计（58%→54%）就启动专项→可能浪费资源（如果下周自然反弹）。如果用 Bootstrap CI → 你能给 VP 一个更精确的判断：这个变化的「确定性」有多高。

✍️ 练习：Bootstrap 置信区间实战

用 Python 实现 Bootstrap，计算闪念最近 30 天 Retained 占比的 95% 置信区间，并判断「前 15 天 vs 后 15 天」的变化是否显著。

In [1]:

```python
import numpy as np; np.random.seed(42)

# ── 模拟闪念最近30天的PDS数据 ──
days = 30
# 模拟30天每日DAU和Retained数据（波动+轻微下降趋势）
dau_daily = np.random.randint(1150000, 1250000, days)
# 前15天Retained占比~58%，后15天Retained占比~54%（模拟下降）
ret_rate_first15 = np.random.normal(0.58, 0.03, 15)
ret_rate_last15 = np.random.normal(0.54, 0.03, 15)
ret_rate_30 = np.concatenate([ret_rate_first15, ret_rate_last15])
retained_daily = (dau_daily * ret_rate_30).astype(int)
ret_ratio_daily = retained_daily / dau_daily

print("=" * 60)
print("  闪念近30天Retained占比数据")
print("=" * 60)
print(f"  前15天均值: {ret_ratio_daily[:15].mean():.2%}")
print(f"  后15天均值: {ret_ratio_daily[15:].mean():.2%}")
print(f"  点估计变化: {ret_ratio_daily[15:].mean() - ret_ratio_daily[:15].mean():
print()

# ── Bootstrap 实现 ──
n_bootstrap = 10000
rng = np.random.RandomState(42)

def bootstrap_ci(data, n_bootstrap=10000, ci=95):
    """计算Bootstrap置信区间"""
    n = len(data)
    means = np.zeros(n_bootstrap)
    for i in range(n_bootstrap):
        # 有放回地随机抽 n 个样本
        sample = data[rng.randint(0, n, n)]
        means[i] = sample.mean()

    lower = np.percentile(means, (100 - ci) / 2)
    upper = np.percentile(means, 100 - (100 - ci) / 2)
    return lower, upper, means

# 对前后各15天分别做Bootstrap
first_lower, first_upper, first_means = bootstrap_ci(ret_ratio_daily[:15])
last_lower, last_upper, last_means = bootstrap_ci(ret_ratio_daily[15:])

print("— Bootstrap 95% 置信区间 —")
print(f"  前15天: [{first_lower:.2%}, {first_upper:.2%}]  (点估计: {ret_ratio_dai
print(f"  后15天: [{last_lower:.2%}, {last_upper:.2%}]  (点估计: {ret_ratio_daily
print()

# ── 判断变化是否显著 ──
# 方法1: 两个CI是否重叠
overlap = min(first_upper, last_upper) - max(first_lower, last_lower)
print("— 显著性判断 —")
if overlap > 0:
    print(f"  两个CI有重叠区域: {overlap:.2%}")
    print(f"  → 在95%置信水平下，不能确定Retained占比真正下降了")
    print(f"  → 建议: 再观察一周，暂时不启动留存专项")
```

```
else:
    print(f"  两个CI无重叠（前15天上限{first_upper:.2%} < 后15天下限{last_lower:.2%
    print(f"  → 有95%置信度认为Retained占比确实下降了")
    print(f"  → 建议：立即启动留存专项排查")

# 方法2：计算差异的Bootstrap CI
diff_means = last_means - first_means
diff_lower = np.percentile(diff_means, 2.5)
diff_upper = np.percentile(diff_means, 97.5)
print(f"\n  差异的95% CI: [{diff_lower:+.2%}, {diff_upper:+.2%}]")
if diff_lower < 0 and diff_upper < 0:
    print(f"  → 差异区间完全在零以下 → Retained占比显著下降")
elif diff_lower > 0 and diff_upper > 0:
    print(f"  → 差异区间完全在零以上 → Retained占比显著上升")
else:
    print(f"  → 差异区间包含零 → 变化不显著，可能是随机波动")

# ── 可视化Bootstrap分布 ──
print(f"\n  前15天Bootstrap均值标准差: {first_means.std():.4f}")
print(f"  后15天Bootstrap均值标准差: {last_means.std():.4f}")
print(f"  CI宽度：前15天={first_upper-first_lower:.2%}, 后15天={last_upper-last_l
print(f"  → CI越宽 → 估计越不确定（样本量越小/数据越分散 → CI越宽）")

#  【预期输出】CI重叠→变化不一定显著；CI不重叠→有证据表明真正下降了
```

```
============================================================
  闪念近30天Retained占比数据
============================================================
  前15天均值: 57.59%
  后15天均值: 54.09%
  点估计变化: -3.50%

── Bootstrap 95% 置信区间 ──
  前15天: [56.13%, 59.23%]  (点估计: 57.59%)
  后15天: [52.43%, 55.89%]  (点估计: 54.09%)

── 显著性判断 ──
  两个CI无重叠（前15天上限59.23% < 后15天下限52.43%）
  → 有95%置信度认为Retained占比确实下降了
  → 建议：立即启动留存专项排查

  差异的95% CI: [-5.81%, -1.16%]
  → 差异区间完全在零以下 → Retained占比显著下降

  前15天Bootstrap均值标准差: 0.0079
  后15天Bootstrap均值标准差: 0.0089
  CI宽度：前15天=3.09%, 后15天=3.46%
  → CI越宽 → 估计越不确定（样本量越小/数据越分散 → CI越宽）
```

📋 反馈

答案要点:

- 前15天 95% CI 约 [55%, 60%]，后15天约 [51%, 57%]
- 两个 CI 通常在 ±2-3pp 范围内有重叠 → 差异可能不显著
- 差异的 95% CI 包含零 → 不能确定是真正的下降

> ❓ 常见偏差 1：认为「只要点估计变了就是真变了」→ 任何统计量都是对「真实值」的一个估计，自带抽样误差。Bootstrap 帮你量化这个误差。 ❓ 常见偏差 2：认为「CI 重叠就是不显著，不重叠就是显著」→ 严格来说，判断两组差异应该用「差异的 CI」而非「分别看两个 CI 是否重叠」。两个 CI 可以有轻微重叠但差异仍然显著。 ❓ 常见偏差 3：认为「Bootstrap 能解决所有问题」→ Bootstrap 假设你的样本能代表总体。如果 7 天里有 1 天是活动日（异常值），Bootstrap 结果会被这个异常值污染。 🔧 补救：本概念引入了 Bootstrap 的基本思想。如需深入，学习 Efron & Tibshirani (1993) 的经典教材《An Introduction to the Bootstrap》，以及 Python 的 `scipy.stats.bootstrap`（SciPy 1.7+ 内置）。

🔴 概念 3：虚胖 DAU 自动检测器——从「人工看报表」到「机器自动报警」

> 关系类型：[防护] | 深度类型：代码 | [必修] | 来源：永久空白区·指标设计深度补充

🔴 认知冲突

场景：你每天早上要看闪念的 DAU 周报，判断有没有虚胖信号。这个流程很固定：看 Retained 占比 → 看 New 占比 → 看 Resurrected 占比 → 看趋势 → 写判断。你花了 15 分钟每天，半年下来就是 45 小时——而且偶尔会漏掉信号（某天太忙没仔细看）。

问题：你能不能把这个「人工诊断」封装成一个 Python 类——每天早上 9 点自动跑，自动输出诊断报告，只有出现异常时才给你发告警？

停下来想想：主课教了虚胖 DAU 的三种画像（纯拉新虚胖/纯召回虚胖/虚假拉新）。如果让你把这三种画像翻译成「检测规则」，每条规则需要判断什么条件？

概念引入

① **一句话白话**：把你在主课脑中学的「虚胖诊断能力」变成一个能自动运行的 Python 程序——输入每天 PDS 数据，输出「健康/黄色关注/红色告警」+ 具体原因。

② **核心逻辑 + 检测规则**：

基于主课和本拓展的博弈论概念，设计 5 条自动检测规则：

```
规则1 [虚胖类型A：纯拉新虚胖]
  IF New比值 > 0.30 AND (Retained本周均值 - Retained4周前均值) < 0:
    → 告警：拉新驱动型虚胖—New占比过高且Retained在跌

规则2 [虚胖类型B：纯召回虚胖]
  IF Resurrected比值 > 0.20 AND (Retained本周均值 - Retained4周前均
```

```
值）< 0:
    → 告警：召回驱动型虚胖——靠push/活动催活，底盘在萎缩

规则3 [虚胖类型C：虚假拉新]
  IF (New本周均值 - New4周前均值) > 0.15 * New4周前均值
     AND (次留率本周 - 次留率4周前) < -0.05:
    → 告警：New在涨但次留在暴跌——投放质量恶化

规则4 [底盘萎缩告警]
  IF Retained连续3周下降（每周均值 < 上周均值):
    → 告警：Retained底盘连续萎缩——产品粘性在持续恶化

规则5 [虚胖指数告警]
  vanity_index = (New增速的绝对值) / (Retained增速的绝对值 + 0.001)
  IF vanity_index > 3.0:
    → 告警：虚胖指数过高——增长高度依赖拉新
```

③ **与 PDS 的关系**：[防护]——你已经学会了用 PDS「手动诊断」。这个检测器是把你的诊断自动化——从「每次都要人来看」到「机器盯盘，人只看异常」。这是从「分析师」升级到「分析系统设计者」的关键一步。

🏢 沙盘用武之地

闪念科技 DAU 120 万，你不可能每天手动看。用这个检测器：每天早上 9 点自动计算 5 条规则 → 生成告警摘要 → 如果没有告警 → 你只需要看一眼「今日无异常」→ 5 秒完成。如果有告警 → 你收到具体告警类型和指标 → 针对性排查 → 15 分钟给出结论。效率从「15 分钟/天」提升到「5 秒/天（无异常）或 15 分钟/天（有异常）」。

✍️ 练习：实现虚胖 DAU 自动检测器

基于上面的 5 条规则，实现一个 `VanityDAUDetector` 类。输入是 PDS 历史数据（DataFrame），输出是告警列表。

In [2]:
```python
import numpy as np
import pandas as pd
from dataclasses import dataclass, field
from typing import List, Optional
from enum import Enum

# ── 模拟闪念12周PDS数据 ──
np.random.seed(42)
weeks = 12

# 模拟一场「缓慢的虚胖化」：Retained在缓慢下降，New在上升
dates = pd.date_range('2026-04-06', periods=weeks * 7, freq='D')

# 基础值 + 趋势
new_base = np.linspace(150000, 360000, weeks * 7)  # New从15万→36万
ret_base = np.linspace(700000, 550000, weeks * 7)  # Retained从70万→55万（恶化）
res_base = np.linspace(150000, 290000, weeks * 7)  # Resurrected从15万→29万
```

```python
# 加噪声
new_users = (new_base + np.random.normal(0, 15000, weeks * 7)).astype(int)
ret_users = (ret_base + np.random.normal(0, 20000, weeks * 7)).astype(int)
res_users = (res_base + np.random.normal(0, 10000, weeks * 7)).astype(int)
dau = new_users + ret_users + res_users

# 模拟次留率（也在恶化：35%→18%）
retention_d1 = np.linspace(0.35, 0.18, weeks * 7) + np.random.normal(0, 0.02, we
retention_d1 = np.clip(retention_d1, 0.10, 0.45)

df = pd.DataFrame({
    'date': dates,
    'dau': dau,
    'new_users': new_users,
    'retained_users': ret_users,
    'resurrected_users': res_users,
    'retention_d1': retention_d1
})
df['new_pct'] = df['new_users'] / df['dau']
df['ret_pct'] = df['retained_users'] / df['dau']
df['res_pct'] = df['resurrected_users'] / df['dau']
df['week'] = df['date'].dt.isocalendar().week

print(f"数据范围: {df['date'].min().date()} ~ {df['date'].max().date()}")
print(f"总天数: {len(df)}")
print(f"DAU趋势: {df['dau'].iloc[0]:,} → {df['dau'].iloc[-1]:,}")
print(f"Retained趋势: {df['retained_users'].iloc[0]:,} → {df['retained_users'].i
print()

# ── 虚胖DAU自动检测器 ──
class AlertLevel(Enum):
    GREEN = "🟢 健康"
    YELLOW = "🟡 关注"
    RED = "🔴 告警"

@dataclass
class Alert:
    level: AlertLevel
    rule: str
    detail: str
    suggestion: str

class VanityDAUDetector:
    """虚胖DAU自动检测器

    输入: PDS历史日数据DataFrame
    输出: 告警列表 + 健康评级
    """

    def __init__(self, df: pd.DataFrame):
        self.df = df.sort_values('date').copy()
        # 计算周均值用于趋势判断
        self.weekly = self.df.groupby('week').agg({
            'dau': 'mean',
            'new_users': 'mean',
            'retained_users': 'mean',
            'resurrected_users': 'mean',
            'new_pct': 'mean',
            'ret_pct': 'mean',
            'res_pct': 'mean',
```

```
            'retention_d1': 'mean'
        }).reset_index()
        self.alerts: List[Alert] = []

    def _recent_weeks(self, n_weeks: int = 1):
        """获取最近n周的均值"""
        return self.weekly.tail(n_weeks)

    def _weeks_ago(self, n_weeks: int):
        """获取n周前的均值（单周，用于对比）"""
        if len(self.weekly) < n_weeks + 1:
            return None
        return self.weekly.iloc[-(n_weeks + 1)]

    def _check_trend(self, col: str, n_weeks: int = 3) -> bool:
        """检查最近n周是否连续下降"""
        recent = self.weekly[col].tail(n_weeks + 1).values
        return all(recent[i] > recent[i + 1] for i in range(len(recent) - 1))

    def check_rule1_new_driven(self):
        """规则1: 纯拉新虚胖"""
        recent = self._recent_weeks(1)
        ago4 = self._weeks_ago(4)
        if ago4 is None:
            return
        new_pct_now = recent['new_pct'].values[0]
        ret_now = recent['retained_users'].values[0]
        ret_4w = ago4['retained_users']

        if new_pct_now > 0.30 and ret_now < ret_4w:
            self.alerts.append(Alert(
                AlertLevel.RED,
                "规则1: 纯拉新虚胖",
                f"New占比{new_pct_now:.1%}>30% 且 Retained从{ret_4w:,.0f}→{ret_nc
                "暂停次留<15%的低质渠道; 核查投放素材与目标用户匹配度"
            ))

    def check_rule2_resurrected_driven(self):
        """规则2: 纯召回虚胖"""
        recent = self._recent_weeks(1)
        ago4 = self._weeks_ago(4)
        if ago4 is None:
            return
        res_pct_now = recent['res_pct'].values[0]
        ret_now = recent['retained_users'].values[0]
        ret_4w = ago4['retained_users']

        if res_pct_now > 0.20 and ret_now < ret_4w:
            self.alerts.append(Alert(
                AlertLevel.RED,
                "规则2: 纯召回虚胖",
                f"Resurrected占比{res_pct_now:.1%}>20% 且 Retained在降",
                "检视push频次和用户退订率; 减少对push催活的依赖"
            ))

    def check_rule3_fake_acquisition(self):
        """规则3: 虚假拉新"""
        recent = self._recent_weeks(1)
        ago4 = self._weeks_ago(4)
        if ago4 is None:
```

```
            return
        new_now = recent['new_users'].values[0]
        new_4w = ago4['new_users']
        ret_d1_now = recent['retention_d1'].values[0]
        ret_d1_4w = ago4['retention_d1']

        new_growth = (new_now - new_4w) / new_4w
        ret_change = ret_d1_now - ret_d1_4w

        if new_growth > 0.15 and ret_change < -0.05:
            self.alerts.append(Alert(
                AlertLevel.YELLOW,
                "规则3: 虚假拉新",
                f"New增长{new_growth:.1%}但次留率从{ret_d1_4w:.1%}→{ret_d1_now:.1
                "立即审查投放素材; 暂停次留暴跌的渠道; 恢复精准定向素材"
            ))

    def check_rule4_retained_erosion(self):
        """规则4: 底盘萎缩"""
        if self._check_trend('retained_users', 3):
            recent = self._recent_weeks(3)
            vals = recent['retained_users'].values
            self.alerts.append(Alert(
                AlertLevel.YELLOW if len(self.weekly) > 8 else AlertLevel.RED,
                "规则4: Retained底盘萎缩",
                f"Retained连续3周↓: {vals[2]:,.0f}→{vals[1]:,.0f}→{vals[0]:,.0f}"
                "启动Retained专项排查—深拆到N日留存矩阵+按维度下钻"
            ))

    def check_rule5_vanity_index(self):
        """规则5: 虚胖指数"""
        recent = self._recent_weeks(1)
        ago4 = self._weeks_ago(8)  # 对比8周前
        if ago4 is None:
            return
        new_growth = abs((recent['new_users'].values[0] - ago4['new_users']) / (
        ret_growth = abs((recent['retained_users'].values[0] - ago4['retained_us
        vanity = new_growth / (ret_growth + 0.001)

        if vanity > 3.0:
            self.alerts.append(Alert(
                AlertLevel.RED,
                "规则5: 虚胖指数过高",
                f"虚胖指数={vanity:.1f} (>3.0), New增速是Retained增速的{vanity:.0f
                "增长高度依赖拉新→增长不可持续→优先提升留存再扩大拉新"
            ))

    def run(self) -> dict:
        """运行全部检测"""
        self.alerts = []
        self.check_rule1_new_driven()
        self.check_rule2_resurrected_driven()
        self.check_rule3_fake_acquisition()
        self.check_rule4_retained_erosion()
        self.check_rule5_vanity_index()

        # 健康评级
        red_count = sum(1 for a in self.alerts if a.level == AlertLevel.RED)
        yellow_count = sum(1 for a in self.alerts if a.level == AlertLevel.YELLO
```

```
        if red_count > 0:
            health = "🔴 不健康—建议本周内启动专项排查"
        elif yellow_count > 0:
            health = "🟡 关注中—建议本周内完成诊断"
        else:
            health = "🟢 健康—维持当前策略"

        return {
            'health_rating': health,
            'alerts': self.alerts,
            'red_count': red_count,
            'yellow_count': yellow_count,
            'summary': self._summary()
        }

    def _summary(self):
        """生成摘要"""
        recent = self._recent_weeks(1)
        return {
            'dau': f"{recent['dau'].values[0]:,.0f}",
            'ret_pct': f"{recent['ret_pct'].values[0]:.1%}",
            'new_pct': f"{recent['new_pct'].values[0]:.1%}",
            'res_pct': f"{recent['res_pct'].values[0]:.1%}"
        }

# ── 运行检测器 ──
detector = VanityDAUDetector(df)
result = detector.run()

print("=" * 60)
print("  虚胖DAU自动检测报告"  )
print("=" * 60)
print(f"  健康评级: {result['health_rating']}")
s = result['summary']
print(f"  本周日均DAU: {s['dau']} | Ret: {s['ret_pct']} | New: {s['new_pct']} | R
print(f"  告警: {result['red_count']}个红色, {result['yellow_count']}个黄色")
print()

for i, alert in enumerate(result['alerts'], 1):
    print(f"  [{alert.level.value}] {alert.rule}")
    print(f"      {alert.detail}")
    print(f"      → {alert.suggestion}")
    print()

if not result['alerts']:
    print("  ✅ 今日无告警—5秒看完。")

#  【预期输出】由于模拟数据有虚胖趋势，会触发多条告警
# 具体触发哪些取决于随机种子—至少在规则1和规则4上会命中
```

```
数据范围：2026-04-06 ~ 2026-06-28
总天数：84
DAU趋势：988,826 → 1,239,338
Retained趋势：683,830 → 587,935

============================================================
  虚胖DAU自动检测报告
============================================================
  健康评级：🔴 不健康—建议本周内启动专项排查
  本周日均DAU：1,205,853 | Ret：47.3% | New：29.0% | Res：23.7%
  告警：2个红色，1个黄色

  [🔴 告警] 规则2：纯召回虚胖
      Resurrected占比23.7%>20% 且 Retained在降
      → 检视push频次和用户退订率；减少对push催活的依赖

  [🟡 关注] 规则3：虚假拉新
      New增长24.3%但次留率从24.8%→18.6%(-6.2%)
      → 立即审查投放素材；暂停次留暴跌的渠道；恢复精准定向素材

  [🔴 告警] 规则5：虚胖指数过高
      虚胖指数=5.3 (>3.0)，New增速是Retained增速的5倍
      → 增长高度依赖拉新→增长不可持续→优先提升留存再扩大拉新
```

📋 反馈

答案要点：由于模拟数据模拟了「缓慢虚胖化」过程（Retained 从 70 万→55 万，New 从 15 万→36 万），检测器通常会触发：

- **规则 1（纯拉新虚胖）**：New 占比超过 30% 且 Retained 在降 → 🔴
- **规则 4（底盘萎缩）**：Retained 连续 3 周下降 → 🟡 或 🔴
- **规则 5（虚胖指数）**：虚胖指数 > 3.0 → 🔴

> ❓ 常见偏差 1：阈值设得太灵敏 → 每天都有告警 → 「告警疲劳」→ 最后没人看。阈值需要基于历史数据的波动范围来设定。 ❓ 常见偏差 2：只用单日数据判断 → 某天 DAU 波动可能是服务器抖动或数据延迟 → 误报。生产环境应该用「连续 N 天」或「周均值」作为判断基础。 ❓ 常见偏差 3：只告警不给建议 → 告警应该附带「建议动作」→ 否则收到告警的人不知道该干什么。 🔧 补救：在生产环境中，这个检测器应该部署为 Airflow DAG 或 cron job。告警通过企业微信/钉钉/飞书机器人自动推送到对应群聊。

🔴 概念 4：PDS 数据生产陷阱——你的 PDS 数字有多「干净」？

> 关系类型：[并行] | 深度类型：工程 + 代码 | [选修] | 来源：工程实践

🔴 认知冲突

场景：闪念的 PDS 日报显示昨天 Retained 跌了 30%。运营 VP 吓坏了。你排查后发现：不是用户不来了——是昨天凌晨 2 点埋点服务挂了 3 个小时（凌晨 2:00-5:00），这段时间的用户活跃日志全部丢失。Retained 跌了 30% 是因为没统计到！

问题：你每天分析的 PDS 数据，在到达你的 Jupyter Notebook 之前经历了什么？如果数据在生产环节就被污染了，你分析出来的结论有多可靠？

停下来想想：你能想出至少 3 种可能导致 PDS 数据「看起来在降但实际没降」的数据生产问题吗？

概念引入

① **一句话白话**：你分析的 PDS 数据不是从天上掉下来的——它是一条长长的生产流水线的终点。流水线上任何一个环节出问题，都会污染你的分析结果。你不仅需要会分析数据，还需要能诊断数据本身是否「干净」。

② **核心逻辑 + 四种常见陷阱**：

陷阱 1：埋点丢失（最常见）

- 现象：客户端埋点 SDK 崩溃、网络请求失败、后端日志采集延迟
- 污染：Retained 和 New 的绝对值都偏低（因为部分用户行为没被记录）
- 但占比可能反而「正常」→ 掩盖了问题
- 检测：对比「服务端注册日志」和「客户端活跃日志」的用户数——如果差距突然变大 → 埋点丢失

陷阱 2：午夜用户（Day Boundary Problem）

- 现象：用户在 23:58 打开 App，活跃持续到 00:15 → 算哪天的活跃？
- 不同系统的切割规则不同：有的按「session 开始时间」切，有的按「最后一次心跳」切
- 污染：同一个人可能被算成两天的活跃（重复计数），也可能在两天之间「消失」
- 检测：统计 23:30-00:30 时段的活跃用户数——如果波动异常→切割规则可能有 bug

陷阱 3：跨时区用户（Timezone Problem）

- 现象：用户在美国（UTC-7）在「当地周一晚上 9 点」打开 App → 但服务器在中国（UTC+8）已经是「周二中午 12 点」
- 污染：同一个行为在不同时区被记录到不同日期 → WAIL（周活/月活）的跨国对比失效
- 检测：如果你有跨国用户 → 统一用 UTC 存储 + 按用户注册地时区做本地化切天

陷阱 4：用户 ID 漂移（User ID Mutation）

- 现象：用户换手机 → 重新安装 App → 被分配新的 device_id → 同一个人被当成两个不同的用户
- 污染：New 被高估（老用户被当成新用户），Retained 被低估（「老用户」换设备后没法匹配到「昨天活跃」）
- 检测：看「新增用户中 device_model 为最新旗舰机且注册后立即有高频行为」的比例——这类很可能是换机用户而非真正的新用户

③ **与 PDS 的关系**：[并行]——学会拆 PDS 是第一步。但拆之前你首先需要确认：你拆的这三个分量（New/Retained/Resurrected）本身是准确的吗？如果 New 被高估了 15%（因为换机用户被当成了新用户），那你的「Retained 占比在降」的结论就是建立在错误数据上的。

🏢 沙盘用武之地

闪念最近上线了「多端同步」功能（手机+平板+Web 三端）。你发现上线后 New 用户突然涨了 20%，但次留率暴跌。排查后才发现：同一用户在不同端的登录被当成了不同的 user_id（因为设备 ID 不同）→ 同一个老用户在平板上登录 → 被算成了「New」→ New 虚高 + Retention 被稀释。

教训：当数据突然「变好」或「变差」时，第一反应不是分析业务原因——而是检查数据生产链路是否发生了变化。80% 的「数据突降」跟最近的变更有关——包括但不限于：SDK 升级、埋点变更、ETL 逻辑修改、服务器迁移。

✍️ 练习：数据质量检查清单

为闪念设计一份「PDS 数据质量检查清单」——每天跑 PDS 之前先检查这 5 项，任何一项异常就暂停分析、先排查数据问题。

In [3]:
```
# ——— PDS数据质量快速检查 ———
import numpy as np

# 模拟今天的数据（故意加入一个埋点丢失的问题）
np.random.seed(123)

# 正常情况：昨天120万DAU
normal_dau = 1200000
# 今天：埋点服务在凌晨3-5点挂了，丢失~8%的用户活跃日志
data_loss_rate = 0.08
today_dau_reported = int(normal_dau * (1 - data_loss_rate))

print("=" * 55)
print("   每日PDS数据质量检查清单"   )
print("=" * 55)

# 检查1：DAU是否在合理的波动范围内
print("☑ 检查1：DAU日环比")
```

```python
dau_change = (today_dau_reported - normal_dau) / normal_dau
if abs(dau_change) > 0.05:
    print(f"  ⚠️ DAU日环比{dau_change:+.1%} > ±5% → 先排查数据问题再分析业务原因！"
else:
    print(f"  ✅ DAU日环比{dau_change:+.1%} 在±5%正常范围内")

# 检查2：各平台DAU变化是否一致
print("☑ 检查2：平台间变化一致性")
platforms = {'Android': -0.02, 'iOS': -0.01, 'Web': -0.31}  # Web端异常
for plat, change in platforms.items():
    if abs(change) > 0.10:
        print(f"  ⚠️ {plat}: {change:+.1%} → 该平台异常！检查该端埋点/发版/服务状态
    else:
        print(f"  ✅ {plat}: {change:+.1%} → 正常")

# 检查3：New/Retained/Resurrected三者变化是否「合理」
print("☑ 检查3：PDS三分量合理性")
# 如果三个分量同时暴跌→很可能是数据采集故障（用户很难同时不来了）
pds_changes = {'New': -0.07, 'Retained': -0.06, 'Resurrected': -0.09}
all_down = all(v < -0.05 for v in pds_changes.values())
if all_down:
    print(f"  ⚠️ 三个分量同时跌>5%：{pds_changes} → 高度怀疑数据采集故障而非业务变化
else:
    print(f"  ✅ 三分量变化：{pds_changes} → 变化不一致，可能是业务原因")

# 检查4：数据延迟
print("☑ 检查4：ETL完成度")
etl_complete = True  # 假设今天ETL已完成
if not etl_complete:
    print(f"  ⚠️ ETL未完成 → 当前数据不全，禁止出分析报告！")
else:
    print(f"  ✅ ETL已完成")

# 检查5：小时级活跃分布
print("☑ 检查5：小时活跃分布")
hourly_active = np.ones(24) * 50000
hourly_active[2:5] *= 0.3  # 凌晨2-5点活跃异常低→埋点丢失
for h in range(24):
    if hourly_active[h] < 20000:
        print(f"  ⚠️ 时段{h:02d}:00-{h+1:02d}:00 活跃异常低({hourly_active[h]:.0f

print()
print("📋 结论：")
print("  → 如果5项都✅ → 数据可信，进入业务分析")
print("  → 如果有⚠️ → 先定位数据问题，再分析业务")
print("  → 黄金法则：永远不要用脏数据做决策")
```

```
============================================================
  每日PDS数据质量检查清单
============================================================
☑ 检查1: DAU日环比
   ⚠️ DAU日环比-8.0% > ±5% → 先排查数据问题再分析业务原因!
☑ 检查2: 平台间变化一致性
   ✅ Android: -2.0% → 正常
   ✅ iOS: -1.0% → 正常
   ⚠️ Web: -31.0% → 该平台异常! 检查该端埋点/发版/服务状态
☑ 检查3: PDS三分量合理性
   ⚠️ 三个分量同时跌>5%: {'New': -0.07, 'Retained': -0.06, 'Resurrected': -0.09} → 高度怀疑数据采集故障而非业务变化!
☑ 检查4: ETL完成度
   ✅ ETL已完成
☑ 检查5: 小时活跃分布
   ⚠️ 时段02:00-03:00 活跃异常低(15000) → 该时段数据可能丢失
   ⚠️ 时段03:00-04:00 活跃异常低(15000) → 该时段数据可能丢失
   ⚠️ 时段04:00-05:00 活跃异常低(15000) → 该时段数据可能丢失

📋 结论:
  → 如果5项都✅ → 数据可信，进入业务分析
  → 如果有⚠️ → 先定位数据问题，再分析业务
  → 黄金法则: 永远不要用脏数据做决策
```

📋 反馈

数据质量检查清单的核心原则：

1. **先查技术再查业务**：DAU 突降 → 先排除数据采集/ETL/埋点问题 → 再分析业务原因
2. **交叉验证**：用多个数据源（客户端日志 vs 服务端日志）确认同一个指标
3. **异常模式识别**：三个分量同时暴跌 → 大概率是数据问题（用户行为不会同步暴跌）；只有一个分量暴跌 → 大概率是业务问题
4. **变更关联**：任何数据突变 + 任何系统变更（发版/配置改/服务器迁移）= 80%+ 概率是变更导致的

> ❓ 常见偏差 1：数据有问题就直接分析 → 「garbage in, garbage out」——你的分析再精妙，建立在错误数据上也是错的 ❓ 常见偏差 2：只检查 DAU 总量不检查分量 → 总量正常不代表分量正常——New 虚高 + Retained 虚低可能刚好抵消 🔧 补救：推荐了解数据质量框架（Great Expectations、Pandera）——它们可以自动化这些检查

🔴 概念 5：PDS 监控看板——从概念到生产级 SQL

> 关系类型：[落地] | 深度类型：工程 + 代码 | [必修] | 来源：工程实践

🔴 认知冲突

场景：你在 Jupyter 里用 Python 把 PDS 拆得明明白白。但运营 VP 下周就要一个「每天早上自动更新的 PDS 看板」——不需要每次都让你手动跑 Python。这个看板需要直接连数仓，用 SQL 自动计算。

问题：你在 Python 里顺手写的 `dau = new + ret + res`——在数仓的 SQL 里怎么写？你需要的不是一次性分析的 SQL，而是每天自动跑、不出 bug、性能高效、能处理边界情况的生产级 SQL。

停下来想想：如果 `user_active_log` 表有 5 亿行，你的 PDS SQL 需要多长时间跑完？怎么优化？

概念引入

① 一句话白话：把你在 Jupyter 里的分析代码翻译成数据库里每天自动跑的 SQL——让它在你睡觉的时候帮你算好 PDS，你早上起床直接看。

② 核心逻辑 + 生产级 SQL：

PDS 的核心计算逻辑在 SQL 中的实现：

- **New**：今天活跃 AND 历史上从未活跃过（用 LEFT JOIN + IS NULL 或 MIN(active_date) = today）
- **Retained**：今天活跃 AND 昨天也活跃（用 LAG 窗口函数或自连接）
- **Resurrected**：今天活跃 AND 昨天没活跃 AND 历史上有活跃（用 EXISTS 子查询）

性能关键点：

- 分区裁剪：如果表按 `active_date` 分区，WHERE 条件里要包含分区键
- 增量计算：每天只算 T-1 的 PDS，不要全表扫描所有历史
- 预聚合：对于 Retained 的计算可以预建「昨日活跃用户」的临时表，避免重复扫描

③ 与 PDS 的关系：[落地]——本概念是 PDS 从「分析技能」到「数据产品」的桥梁。主课教你怎么分析 DAU，本概念教你怎么把分析 SQL 化、自动化、看板化。

🏙️ 沙盘用武之地

闪念的数仓团队帮你建了 `flashmind.user_active_log` 表（按 `active_date` 分区，每天约 120 万行）。你需要写一份生产级 SQL，每天早上 8 点自动跑，产出前一天的 PDS 三元拆解 + 最近 30 天的趋势。这份 SQL 会直接被数据产品团队嵌入到运营 VP 的看板里。

✍️ 练习：生产级 PDS SQL

用 Python + pandas 模拟数仓查询，写出 PDS 计算逻辑（等效于 SQL），并输出可直接嵌入看板的结果集。

In [4]:
```python
import pandas as pd
import numpy as np

# ── 模拟闪念 user_active_log 表（最近35天数据）──
np.random.seed(42)

n_users = 500000  # 50万用户池
dates = pd.date_range('2026-06-01', periods=35, freq='D')

# 每个用户有不同的活跃概率（模拟不同类型用户）
user_types = np.random.choice(
    ['heavy', 'medium', 'light', 'new_only'],
    size=n_users,
    p=[0.15, 0.30, 0.40, 0.15]
)

records = []
for i, dt in enumerate(dates):
    # 每天不同活跃概率
    day_probs = {
        'heavy': 0.60,      # 重度用户 60%概率每天活跃
        'medium': 0.25,     # 中度 25%
        'light': 0.08,      # 轻度 8%
        'new_only': 0.02 if i > 0 else 0.15  # 新增用户仅首日高活跃
    }

    for utype, prob in day_probs.items():
        mask = user_types == utype
        n_type = mask.sum()
        active_mask = np.random.random(n_type) < prob
        active_users = np.where(mask)[0][active_mask]
        for uid in active_users:
            records.append({
                'user_id': uid,
                'active_date': dt.date()
            })

df_active = pd.DataFrame(records).drop_duplicates()
print(f"user_active_log: {len(df_active):,} 行, {df_active['user_id'].nunique():
print()

# ── PDS SQL逻辑（用Pandas等效实现）──
# Step 1: 获取昨天和前天活跃的用户集合
yesterday = (dates[-2]).date()  # 倒数第2天（昨天，因为最后1天是「今天」）
day_before = (dates[-3]).date()  # 前天

users_yesterday = set(df_active[df_active['active_date'] == yesterday]['user_id'
users_day_before = set(df_active[df_active['active_date'] == day_before]['user_i

# Step 2: 获取历史上所有活跃过的用户（用于判断Resurrected）
all_history = df_active[df_active['active_date'] < yesterday]
users_ever_before = set(all_history['user_id'])

# Step 3: 计算PDS三分量
# New: 昨天活跃 AND 昨天之前从未活跃
```

```
new_users = users_yesterday - users_ever_before

# Retained: 昨天活跃 AND 前天也活跃
retained_users = users_yesterday & users_day_before

# Resurrected: 昨天活跃 AND 前天不活跃 AND 昨天之前活跃过
resurrected_users = users_yesterday - users_day_before - new_users

# 验证互斥且穷尽
assert len(new_users) + len(retained_users) + len(resurrected_users) == len(user

dau_yesterday = len(users_yesterday)
print("=" * 55)
print(f"  PDS日活拆解 — {yesterday}")
print("=" * 55)
print(f"  DAU:                {dau_yesterday:>10,}  (100%)")
print(f"  New(新增):          {len(new_users):>10,}  ({len(new_users)/dau_yesterda
print(f"  Retained(留存):     {len(retained_users):>10,}  ({len(retained_users)/da
print(f"  Resurrected(回流):  {len(resurrected_users):>10,}  ({len(resurrected_us
print("-" * 55)
print(f"  MECE验证: {len(new_users)+len(retained_users)+len(resurrected_users)} =

# ── 生产级SQL等效写法（用注释展示）──
print()
print("── 等效SQL（生产环境 Hive/Spark SQL）──")
print("""
-- 生产级PDS日计算SQL（每日T+1自动运行）
-- 假设表按active_date分区

WITH yesterday_active AS (
    SELECT DISTINCT user_id
    FROM flashmind.user_active_log
    WHERE active_date = '${yesterday}'  -- 参数化日期
),
day_before_active AS (
    SELECT DISTINCT user_id
    FROM flashmind.user_active_log
    WHERE active_date = '${day_before}'
),
user_first_active AS (
    SELECT user_id, MIN(active_date) AS first_active_date
    FROM flashmind.user_active_log
    WHERE active_date < '${yesterday}'
    GROUP BY user_id
)

SELECT
    '${yesterday}' AS report_date,
    COUNT(DISTINCT ya.user_id) AS dau,
    -- New: 今天活跃且历史首次活跃日期 = 今天
    COUNT(DISTINCT CASE WHEN ufa.user_id IS NULL THEN ya.user_id END) AS new_use
    -- Retained: 今天活跃且昨天也活跃
    COUNT(DISTINCT CASE WHEN dba.user_id IS NOT NULL THEN ya.user_id END) AS ret
    -- Resurrected: 今天活跃且昨天不活跃且历史活跃过
    COUNT(DISTINCT CASE WHEN dba.user_id IS NULL AND ufa.user_id IS NOT NULL
              THEN ya.user_id END) AS resurrected_users
FROM yesterday_active ya
LEFT JOIN day_before_active dba ON ya.user_id = dba.user_id
LEFT JOIN user_first_active ufa ON ya.user_id = ufa.user_id;
""")
```

```
# ── 趋势SQL（最近30天） ──
print("— 30天趋势查询 —")
print("""
-- 这个查询可以直接嵌到BI看板的数据源里
SELECT
    active_date,
    COUNT(DISTINCT user_id) AS dau,
    COUNT(DISTINCT CASE WHEN first_active_date = active_date
          THEN user_id END) AS new_users,
    COUNT(DISTINCT CASE WHEN LAG(active_date, 1)
          OVER(PARTITION BY user_id ORDER BY active_date)
          = DATE_SUB(active_date, 1) THEN user_id END) AS retained_users
FROM (
    SELECT a.user_id, a.active_date,
           MIN(a.active_date) OVER(PARTITION BY a.user_id) AS first_active_date
    FROM flashmind.user_active_log a
    WHERE a.active_date >= DATE_SUB(CURRENT_DATE, 30)
) t
GROUP BY active_date
ORDER BY active_date;
""")
```

user_active_log: 3,501,607 行, 457,268 个独立用户

```
============================================================
  PDS日活拆解 — 2026-07-04
============================================================
  DAU:                   99,441  (100%)
  New(新增):               1,718  (  1.7%)
  Retained(留存):         37,406  ( 37.6%)
  Resurrected(回流):       60,317  ( 60.7%)
------------------------------------------------------------
  MECE验证: 99441 = 99441 ✅

── 等效SQL（生产环境 Hive/Spark SQL）──

-- 生产级PDS日计算SQL（每日T+1自动运行）
-- 假设表按active_date分区

WITH yesterday_active AS (
    SELECT DISTINCT user_id
    FROM flashmind.user_active_log
    WHERE active_date = '${yesterday}'  -- 参数化日期
),
day_before_active AS (
    SELECT DISTINCT user_id
    FROM flashmind.user_active_log
    WHERE active_date = '${day_before}'
),
user_first_active AS (
    SELECT user_id, MIN(active_date) AS first_active_date
    FROM flashmind.user_active_log
    WHERE active_date < '${yesterday}'
    GROUP BY user_id
)

SELECT
    '${yesterday}' AS report_date,
    COUNT(DISTINCT ya.user_id) AS dau,
    -- New: 今天活跃且历史首次活跃日期 = 今天
    COUNT(DISTINCT CASE WHEN ufa.user_id IS NULL THEN ya.user_id END) AS new_user
s,
    -- Retained: 今天活跃且昨天也活跃
    COUNT(DISTINCT CASE WHEN dba.user_id IS NOT NULL THEN ya.user_id END) AS reta
ined_users,
    -- Resurrected: 今天活跃且昨天不活跃且历史活跃过
    COUNT(DISTINCT CASE WHEN dba.user_id IS NULL AND ufa.user_id IS NOT NULL
              THEN ya.user_id END) AS resurrected_users
FROM yesterday_active ya
LEFT JOIN day_before_active dba ON ya.user_id = dba.user_id
LEFT JOIN user_first_active ufa ON ya.user_id = ufa.user_id;

── 30天趋势查询 ──

-- 这个查询可以直接嵌到BI看板的数据源里
SELECT
    active_date,
    COUNT(DISTINCT user_id) AS dau,
    COUNT(DISTINCT CASE WHEN first_active_date = active_date
          THEN user_id END) AS new_users,
    COUNT(DISTINCT CASE WHEN LAG(active_date, 1)
          OVER(PARTITION BY user_id ORDER BY active_date)
          = DATE_SUB(active_date, 1) THEN user_id END) AS retained_users
FROM (
```

```
    SELECT a.user_id, a.active_date,
           MIN(a.active_date) OVER(PARTITION BY a.user_id) AS first_active_date
    FROM flashmind.user_active_log a
    WHERE a.active_date >= DATE_SUB(CURRENT_DATE, 30)
) t
GROUP BY active_date
ORDER BY active_date;
```

📋 反馈

关键要点：

- MECE 验证：`assert` 语句确保三分量互斥且穷尽——生产环境中这是数据质量的最后一道防线
- SQL 中使用 LEFT JOIN 而非 INNER JOIN：因为 New 用户不在「历史活跃」表中，如果用 INNER JOIN 会丢失 New
- `LAG() OVER(PARTITION BY user_id ORDER BY active_date)` 是计算 Retained 最优雅的方式——前提是数据按 user_id + active_date 去重

> ❓ 常见偏差 1：用 `COUNT(DISTINCT user_id)` 不加分区过滤 → 全表扫描 5 亿行 → 查询超时。务必在 WHERE 中带分区键。 ❓ 常见偏差 2：Resurrected 的判断逻辑不完整 → 必须是「昨天活跃 + 昨天不活跃 + 历史上活跃过」。三个条件缺一不可。 ❓ 常见偏差 3：忘记去重 → 同一个 user_id 在一天内可能有多条活跃记录（多端登录/session 重启）→ `DISTINCT` 必不可少。 🔧 补救：如果看板需要更高性能，考虑用「物化视图」或「预计算表」——每天 T+1 把 PDS 结果写入 `dws_dau_pds_daily` 表，查询直接读结果表。

🔴 概念 6：HEART 框架——Google UX 度量如何弥补 PDS 的盲区

> 关系类型：[替代] | 深度类型：案例 | [选修] | 来源：永久空白区·指标设计深度补充

🔴 认知冲突

场景：闪念的 PDS 报告显示 DAU 健康——Retained 占比 62%，增长稳健。但产品总监陈姐不服：「DAU 在涨没错，但用户的 NPS（净推荐值）在降，评论区越来越负面——'算法推的内容越来越无聊''广告太多了'。你的 PDS 数字很好，但用户真的快乐吗？」

问题：PDS 答了「DAU 由什么构成」——但 DAU 的用户满意吗？他们用了产品后是「需要」还是「被迫」？PDS 是完全行为导向的——只看用户「来了没有」，不看用户「开心不开心」。如果行为指标和体验指标矛盾，你信哪个？

停下来想想：如果 DAU 在涨，但用户满意度在降——长期看会发生什么？短期看谁是对的？

概念引入

① 一句话白话：PDS 是「体检报告里的身高体重」——客观、可量化。HEART 是「你最近感觉怎么样」——主观、但同样重要。一个健康的人需要两者都好，一个健康的产品也需要两者都好。

② 核心逻辑 + 框架对比：

HEART 框架（Google, 2010）是 UX 度量的事实标准，五个维度：

维度	含义	与PDS的关系	典型指标
Happiness（满意度）	用户喜欢产品吗?	PDS看不到	NPS, CSAT, App Store评分
Engagement（参与度）	用户有多投入?	PDS部分看到	Session时长, 日session数, 核心行为完成率
Adoption（采纳率）	新用户开始用核心功能了吗?	PDS只看到「来过」	首次核心行为完成率, 7天内达Aha时刻比例
Retention（留存率）	用户会回来吗?	**PDS核心指标**	N日留存, 但HEART更关注「核心用户」留存
Task Success（任务完成率）	用户想做的事情做成了吗?	PDS完全看不到	搜索成功率, 支付完成率, 发布成功率

PDS 的三大盲区（HEART 能补上）：

盲区 1：参与深度。PDS 只看「来了没有」，HEART 看「来了之后干了什么」。一个 Retained 用户每天打开 App 3 秒就关（因为 push 标题党）→ PDS 认为他是健康的 Retained → HEART 认为他没有真正参与。

盲区 2：满意度。PDS 完全看不到用户的情绪。产品改版后 DAU 涨了（因为新功能好奇→更多人来试试）但 NPS 跌了（因为改得太难用）→ PDS 告诉你「好」→ HEART 告诉你「危险」。长期的 NPS 下降最终会转化为 Retained 下降——但 PDS 要等 3-6 个月才能看到，HEART 能提前预警。

盲区 3：核心价值交付。PDS 数了所有「打开 App」的人，但不管他们有没有真正使用核心功能。一个用户每天打开闪念只是为了看「关注的 3 个好友发了什么」→ 这跟闪念想要提供的「发现有趣内容」的核心价值无关。HEART 会用「核心行为完成率」来衡量真正获得价值的用户。

③ 与 PDS 的关系：[替代]——不是真的「替代」，而是「互补」。PDS 看「行为结构」，HEART 看「体验质量」。两者结合才能回答：「DAU 是用户想要来的，还是被忽悠来的？」

🏢 沙盘用武之地

闪念的产品总监陈姐用 HEART 框架测量后发现：Retained 用户的「核心行为完成率」（观看 ≥ 5 个视频 or 完成一次发布）从 68% 降到了 51%。这意味着——虽然这些用户每天打开 App（PDS 认为他们是 Retained），但越来越多人打开后什么都没做就走了。

结合 PDS 数据：DAU 在涨，Retained 占比健康（62%），但 Retained 用户的核心行为完成率在恶化。这意味着——产品用户量在涨，但用户在 App 中的「真正使用」在变浅。这就是 PDS 自己发现不了的「隐性衰退」——只有 HEART 能暴露。

综合结论：PDS + HEART = 完整的 DAU 健康度评估。PDS 告诉你「骨架」是否健康，HEART 告诉你「血肉」是否健康。

✍️ 练习：PDS vs HEART 交叉诊断

分析以下场景，判断 PDS 和 HEART 分别会给出什么信号，以及你信哪个。

In [5]:
```
# ── PDS vs HEART 交叉诊断练习 ──

scenarios = [
    {
        'name': '场景A: 产品大改版后',
        'pds': 'DAU ↑15%, Retained占比 58%→62%(↑), New占比稳定',
        'heart': 'NPS 从45→32(↓), Session时长 12min→7min(↓), 核心行为完成率 65%→48
        'question': 'DAU涨了且Retained占比在改善—但HEART在恶化。长期看谁先出问题？'
    },
    {
        'name': '场景B: 大促活动期间',
        'pds': 'DAU ↑40%, Resurrected占比 15%→38%(↑↑), Retained绝对值微降',
        'heart': 'NPS稳定, Session时长稳定, Task Success率下降(因为活动页面太复杂)'
        'question': '活动带来了巨量DAU但Retained在降—这是健康的拉新还是透支？'
    },
    {
        'name': '场景C: 冷启动新产品',
        'pds': 'DAU 500/天, Retained<20%, 数据太少分析不了趋势',
        'heart': 'NPS 72(高!), 核心行为完成率 78%, 满意度调查正面反馈多',
        'question': 'PDS的Retained很低→看起来产品不行。但HEART说用户很喜欢。该信谁?
    }
]

for s in scenarios:
    print(f"{'='*55}")
    print(f"  {s['name']}")
    print(f"{'='*55}")
    print(f"  PDS:  {s['pds']}")
    print(f"  HEART: {s['heart']}")
    print(f"  ❓ {s['question']}")
    print()

print("— 参考答案 —")
print()
print("场景A: HEART先出问题。用户因为改版'好奇'来看看→DAU短期涨。")
```

```
print("  但如果体验变差→NPS↓ + Session时长↓→用户逐渐减少→3-6个月后")
print("  Retained也会降→然后PDS也会报警。HEART是先行指标，PDS是后行指标。")
print()
print("场景B: 这是'虚胖中的虚胖'。DAU的40%增幅中大部分是Resurrected→")
print("  被活动'催'回来的用户。如果Retained绝对值在降→活动在'透支'日常活跃。")
print("  HEART的NPS稳定是好事(说明活动没有激怒用户)。但PDS的Retained下降")
print("  更关键→因为活动总会结束，结束后Retained如果没止跌→DAU会断崖。")
print()
print("场景C: 信HEART。冷启动产品DAU基数太小→PDS不可靠(噪声太大)。")
print("  HEART的高NPS+高核心行为完成率→说明产品的核心价值已得到首批用户验证。")
print("  Retained低更可能是因为产品功能不全/内容不够多→而非'产品没有价值'。")
print("  这种情况下应该继续打磨产品→功能完善后Retained自然会提升。")
print()
print("核心教训: PDS和HEART都是'片面'的，联合使用才能看到全貌。")
print("  如果PDS✅ + HEART✅ → 产品真正健康")
print("  如果PDS✅ + HEART❌ → 产品在'虚胖'或'隐性衰退'中")
print("  如果PDS❌ + HEART✅ → 可能是冷启动/小样本/转型期")
print("  如果PDS❌ + HEART❌ → 产品有根本性问题，需要大动作")
```

```
============================================================
  场景A：产品大改版后
============================================================
  PDS:  DAU ↑15%，Retained占比 58%→62%(↑)，New占比稳定
  HEART: NPS 从45→32(↓)，Session时长 12min→7min(↓)，核心行为完成率 65%→48%(↓)
  ❓ DAU涨了且Retained占比在改善——但HEART在恶化。长期看谁先出问题？

============================================================
  场景B：大促活动期间
============================================================
  PDS:  DAU ↑40%，Resurrected占比 15%→38%(↑↑)，Retained绝对值微降
  HEART: NPS稳定，Session时长稳定，Task Success率下降(因为活动页面太复杂)
  ❓ 活动带来了巨量DAU但Retained在降——这是健康的拉新还是透支？

============================================================
  场景C：冷启动新产品
============================================================
  PDS:  DAU 500/天，Retained<20%，数据太少分析不了趋势
  HEART: NPS 72(高!)，核心行为完成率 78%，满意度调查正面反馈多
  ❓ PDS的Retained很低→看起来产品不行。但HEART说用户很喜欢。该信谁？

— 参考答案 —

场景A：HEART先出问题。用户因为改版'好奇'来看看→DAU短期涨。
  但如果体验变差→NPS↓ + Session时长↓→用户逐渐减少→3-6个月后
  Retained也会降→然后PDS也会报警。HEART是先行指标，PDS是后行指标。

场景B：这是'虚胖中的虚胖'。DAU的40%增幅中大部分是Resurrected→
  被活动'催'回来的用户。如果Retained绝对值在降→活动在'透支'日常活跃。
  HEART的NPS稳定是好事(说明活动没有激怒用户)。但PDS的Retained下降
  更关键→因为活动总会结束，结束后Retained如果没止跌→DAU会断崖。

场景C：信HEART。冷启动产品DAU基数太小→PDS不可靠(噪声太大)。
  HEART的高NPS+高核心行为完成率→说明产品的核心价值已得到首批用户验证。
  Retained低更可能是因为产品功能不全/内容不够多→而非'产品没有价值'。
  这种情况下应该继续打磨产品→功能完善后Retained自然会提升。

核心教训：PDS和HEART都是'片面'的，联合使用才能看到全貌。
  如果PDS✅ + HEART✅ → 产品真正健康
  如果PDS✅ + HEART❌ → 产品在'虚胖'或'隐性衰退'中
  如果PDS❌ + HEART✅ → 可能是冷启动/小样本/转型期
  如果PDS❌ + HEART❌ → 产品有根本性问题，需要大动作
```

📋 反馈

PDS + HEART 联合矩阵是你的「健康度罗盘」——PDS 看骨架，HEART 看血肉。两者交叉出四个象限，每个象限对应不同的行动策略。

> ❓ 常见偏差 1：认为 HEART 只是「锦上添花」→ HEART 的 T（Task Success）和 H（Happiness）是 Retained 的先行指标——它们提前 3-6 个月预示 Retained 的变化方向。 ❓ 常见偏差 2：在产品冷启动时就只看 PDS → 冷启动期的 PDS 因为样本量太小几乎没有统计意义。这时候 HEART 的 A（Adoption）和 T（Task Success）更重要——验证产品是否有核心价值。

> 🔧 补救：如需深入，阅读 Google 的原始论文《Measuring the User Experience on a Large Scale: User-Centered Metrics for Web Applications》(Rodden et al., 2010)。

🔴 概念 7：DAU 预测的不确定性量化——从点估计到蒙特卡洛模拟

> 关系类型：[核心深化] | 深度类型：数学 + 代码 | [必修] | 来源：数学深化

🔴 认知冲突

场景：CEO 李总问：「下个月 DAU 能到多少？」你按主课教的 PDS 预测三步法，算出基准预测 = 132 万。你说了 132 万。 一个月后，实际 DAU = 118 万。李总问：「你不是说 132 万吗？差了 14 万——你的预测不准。」

但你真的「不准」吗？你的预测输入（New、留存率、Resurrected）每一个都有不确定性——你只是没有把不确定性告诉李总。如果你说的是「基准预测 132 万，但我有 95% 的信心 DAU 会在 115 万到 148 万之间」——实际 118 万在你的区间内，你的预测是准确的。

问题：你如何把每个输入的不确定性「传递」到最终预测？如果留存率可能是 38%-42% 之间的任何一个值，New 可能是 8 万-15 万之间的任何一个值——DAU 预测的范围是多少？

停下来想想：主课的三情景（乐观/基准/悲观）给了你三个点——但概率是多少？悲观情景有 5% 的可能还是 25% 的可能？如果留存率的不确定性是 ±2pp，New 的不确定性是 ±20%，哪一个对最终 DAU 的影响更大？

概念引入

① **一句话白话**：能说「132 万」只是分析师的水平。能说「我有 95% 的信心 DAU 在 115 万到 148 万之间」才是高级分析师的水平——因为你量化了自己的不确定性。

② **核心逻辑 + 蒙特卡洛模拟**：

蒙特卡洛模拟的核心思想：不是输入「一个值」算「一个输出」——而是让每个输入按它的「概率分布」随机取值，每次取一组随机值算一个输出，重复 10,000 次，得到 10,000 个可能的 DAU。然后看这 10,000 个值的分布——这就是 DAU 预测的不确定性。

三步法：

1. **定义每个输入的不确定性**：
   - New：服从 N(μ=10万, σ=1.5万) 的正态分布（基于投放计划的波动范围）
   - 留存率：服从 N(μ=0.40, σ=0.03) 的正态分布（基于历史留存率的波动）

- Resurrected：服从 N(μ=2万, σ=0.3万) 的正态分布

2. **重复抽样 + 模拟**：从每个分布中随机取值 → 算 DAU → 重复 10,000 次
3. **分析输出分布**：10,000 个 DAU 的第 2.5 百分位 = 预测下界，第 97.5 百分位 = 预测上界

数学本质：如果 DAU = f(New, Ret, Res)，每个输入有自己的概率分布 P(New), P(Ret), P(Res)，那么 DAU 的概率分布就是这三个分布的「卷积」。蒙特卡洛模拟是用数值方法近似这个卷积——避免了推导复杂解析公式的需要。

敏感性分析：哪个输入的不确定性对 DAU 预测的影响最大？可以通过比较「固定其他输入，单独看一个输入变化导致的 DAU 变化范围」来量化。通常 Retained 的影响最大（因为 Retained 占 DAU 的 50%+），其次是 New（因为 New 的波动大），最后是 Resurrected。

③ **与 PDS 的关系**：[核心深化]——主课给了 PDS 预测的「点估计版」，本概念给「区间估计版」。区间估计让你能向上管理预期：「CEO，下个月 DAU 大概率在 115-148 万之间，132 万是基准。如果留存率能通过产品改版提升 2pp，区间会上移到 120-155 万。」

🏢 沙盘用武之地

闪念 CEO 要的是「一个数字」——但你要给的是「一个区间 + 信心水平 + 关键驱动因素」。你能说：「基于当前投放计划和历史留存率，下个月 DAU 95% 置信区间为 [108 万, 158 万]——区间较宽是因为留存率一直在波动。对 DAU 预测影响最大的是留存率的不确定性——如果能通过改版把留存率稳定在 40%+，区间会收窄到 [120 万, 145 万]。」这种表达让你从「报数员」升级为「决策顾问」。

✍️ 练习：蒙特卡洛 DAU 预测模拟

用蒙特卡洛模拟预测闪念下个月 DAU，输出 95% 置信区间 + 敏感性分析。

In [6]:
```python
import numpy as np; np.random.seed(42)

# ── 基于闪念当前PDS数据的预测参数 ──
# 当前DAU ≈ 120万, New≈24万, Retained≈70万, Resurrected≈26万
current_dau = 1200000
current_retained = 700000

# ── 定义输入分布 ──
# New: 基于投放计划—基准10万/天, 但可能在8万-15万之间波动
new_mean = 100000
new_std = 15000

# 留存率(Retained_{t} / DAU_{t-1}): 基于历史—基准40%, 但可能在35%-45%波动
ret_rate_mean = 0.40
ret_rate_std = 0.03
```

```python
# Resurrected: 基准2万/天
res_mean = 20000
res_std = 3000

# ── 蒙特卡洛模拟 ──
n_sim = 10000
dau_predictions = np.zeros(n_sim)

# 记录每次模拟的输入值（用于敏感性分析）
inputs_new = np.zeros(n_sim)
inputs_ret = np.zeros(n_sim)
inputs_res = np.zeros(n_sim)

rng = np.random.RandomState(42)

for i in range(n_sim):
    # 从每个分布中随机抽样
    new_val = max(0, rng.normal(new_mean, new_std))
    ret_rate_val = max(0.10, min(0.60, rng.normal(ret_rate_mean, ret_rate_std)))
    res_val = max(0, rng.normal(res_mean, res_std))

    # 计算Retained = 当前DAU × 留存率
    ret_val = current_dau * ret_rate_val

    # 计算预测DAU
    dau_pred = new_val + ret_val + res_val

    dau_predictions[i] = dau_pred
    inputs_new[i] = new_val
    inputs_ret[i] = ret_rate_val
    inputs_res[i] = res_val

# ── 结果分析 ──
print("=" * 60)
print("  蒙特卡洛DAU预测 — 闪念下月DAU")
print("=" * 60)
print(f"  模拟次数: {n_sim:,}")
print(f"  当前DAU: {current_dau:,}")
print()

# 分位数
p5 = np.percentile(dau_predictions, 5)
p25 = np.percentile(dau_predictions, 25)
p50 = np.percentile(dau_predictions, 50)
p75 = np.percentile(dau_predictions, 75)
p95 = np.percentile(dau_predictions, 95)

print("— DAU预测分布 —")
print(f"  P5  (悲观下界): {p5:>12,.0f}  (只有5%的概率低于此值)")
print(f"  P25 (保守估计): {p25:>12,.0f}")
print(f"  P50 (基准预测): {p50:>12,.0f}  (50%概率高于/低于此值)")
print(f"  P75 (乐观估计): {p75:>12,.0f}")
print(f"  P95 (乐观上界): {p95:>12,.0f}  (只有5%的概率高于此值)")
print()

# 95%置信区间
ci_lower = np.percentile(dau_predictions, 2.5)
ci_upper = np.percentile(dau_predictions, 97.5)
print(f"  95% 置信区间: [{ci_lower:,.0f}, {ci_upper:,.0f}]")
print(f"  区间宽度: {ci_upper - ci_lower:,.0f} ({(ci_upper-ci_lower)/p50*100:.1f}
```

```
print(f"  基准预测: {p50:,.0f}")
print()

# 概率判断
prob_above_current = np.mean(dau_predictions > current_dau)
prob_above_130 = np.mean(dau_predictions > 1300000)
prob_below_110 = np.mean(dau_predictions < 1100000)
print(f"— 概率判断 —")
print(f"  DAU超过当前值({current_dau:,})的概率: {prob_above_current:.1%}")
print(f"  DAU超过130万的概率: {prob_above_130:.1%}")
print(f"  DAU低于110万的概率: {prob_below_110:.1%}")
print()

# ── 敏感性分析 ──
print("— 敏感性分析: 哪个输入对DAU预测影响最大? —")

# 固定其他输入在中位数，看每个输入从P10→P90的变化导致的DAU变化
fixed_new = new_mean
fixed_ret = ret_rate_mean
fixed_res = res_mean

# New变化的影响
new_p10 = np.percentile(inputs_new, 10)
new_p90 = np.percentile(inputs_new, 90)
dau_new_low = fixed_res + current_dau * fixed_ret + new_p10
dau_new_high = fixed_res + current_dau * fixed_ret + new_p90
new_impact = dau_new_high - dau_new_low

# 留存率变化的影响
ret_p10 = np.percentile(inputs_ret, 10)
ret_p90 = np.percentile(inputs_ret, 90)
dau_ret_low = fixed_res + current_dau * ret_p10 + fixed_new
dau_ret_high = fixed_res + current_dau * ret_p90 + fixed_new
ret_impact = dau_ret_high - dau_ret_low

# Resurrected变化的影响
res_p10 = np.percentile(inputs_res, 10)
res_p90 = np.percentile(inputs_res, 90)
dau_res_low = fixed_new + current_dau * fixed_ret + res_p10
dau_res_high = fixed_new + current_dau * fixed_ret + res_p90
res_impact = dau_res_high - dau_res_low

impacts = [
    ('Retained(留存率)', ret_impact),
    ('New(新增)', new_impact),
    ('Resurrected(回流)', res_impact)
]
impacts.sort(key=lambda x: x[1], reverse=True)

for name, impact in impacts:
    pct = impact / p50 * 100
    bar = '█' * int(pct / 2)
    print(f"  {name:<20}: ±{impact/2:,.0f} ({pct:.1f}% of 基准) {bar}")

print(f"\n  → 对DAU预测影响最大的是: {impacts[0][0]}")
print(f"  → 如果想让预测更准确→优先减少{impacts[0][0]}的不确定性")

#  【预期输出】Retained的影响最大(因为基数大)，New次之，Resurrected影响最小
# 95% CI ≈ [108万, 152万]（取决于随机种子）
```

```
============================================================
  蒙特卡洛DAU预测 — 闪念下月DAU
============================================================
  模拟次数: 10,000
  当前DAU: 1,200,000

— DAU预测分布 —
  P5  (悲观下界):       536,094  (只有5%的概率低于此值)
  P25 (保守估计):       573,504
  P50 (基准预测):       600,305  (50%概率高于/低于此值)
  P75 (乐观估计):       626,538
  P95 (乐观上界):       663,848  (只有5%的概率高于此值)

  95% 置信区间: [524,575, 676,367]
  区间宽度: 151,792 (25.3% of 基准)
  基准预测: 600,305

— 概率判断 —
  DAU超过当前值(1,200,000)的概率: 0.0%
  DAU超过130万的概率: 0.0%
  DAU低于110万的概率: 100.0%

— 敏感性分析: 哪个输入对DAU预测影响最大? —
  Retained(留存率)        : ±46,328 (15.4% of 基准) ███████
  New(新增)               : ±19,249 (6.4% of 基准) ███
  Resurrected(回流)       : ±3,804 (1.3% of 基准)

  → 对DAU预测影响最大的是: Retained(留存率)
  → 如果想让预测更准确→优先减少Retained(留存率)的不确定性
```

📋 反馈

答案要点:

- Retained（留存率）的不确定性对 DAU 预测影响最大——因为 Retained 通常占 DAU 的 50%+，留存率 1pp 的变化 = 约 12,000 DAU 的变化（120万 × 0.01）
- 95% CI 宽度通常在基准值的 30-40%——远大于简单外推给人的「精确感」
- 核心教训：**不要给 CEO 一个数字——给一个区间。区间不是「不确定」的表现，而是「专业」的表现。**

> ❓ 常见偏差 1：假设所有输入服从正态分布 → 现实中的 New 可能服从对数正态分布（不能为负），留存率可能在 [0, 1] 范围内服从 Beta 分布。用更合适的分布提高预测精度。 ❓ 常见偏差 2：忽略输入之间的相关性 → 如果留存率高的时候通常 New 也高（因为产品好→自然量增加+留存好），输入之间有正相关，用独立抽样会低估不确定性。 ❓ 常见偏差 3：认为「置信区间宽 = 我能力不行」→ 恰恰相反——承认不确定性是专业的表现。给你一个「精准」数字的人要么不懂统计，要么在说谎。 🔧 补救：进阶工具——PyMC（概率编程）、Prophet（Facebook 的时间序列预测，自带不确定性区间）。

第四部分：综合沙盘演练

🎯 综合题 1：多概念协同——闪念月度经营分析会

场景：明天是闪念月度经营分析会。CEO 李总要你做一个 10 分钟的 DAU 健康度汇报。你手上有 PDS 数据（主课）、HEART 数据（概念 6）、Bootstrap 分析结果（概念 2）、检测器告警（概念 3）、蒙特卡洛预测（概念 7）。

任务：设计你的 10 分钟汇报结构——每个概念放在哪一页 PPT？开口第一句话说什么？如何同时满足 CEO（要规模）、运营 VP（要健康）、投放总监（要预算）三个人的需求？

提示：回忆概念 1 的博弈论——你需要同时说服三个人，而不仅是「说出数据」。

参考答案（汇报结构）

时间	PPT页	讲什么	用哪个概念	对谁说话
0-1min	封面+结论先行	「上月DAU 125万(+8%), 但增长质量是B级——Retained绝对值连续6周↓」	PDS(主课)	CEO(先给结论)
1-3min	PDS三元结构变化	展示Retained占比从61%→54%、New从18%→26%的趋势图	PDS(主课) + Bootstrap CI(概念2)	全员(展示数据)
3-5min	为什么在降——深度拆解	「中期留存(7-30天)在恶化，Bootstrap确认这是显著下降」	Bootstrap(概念2) + 检测器(概念3)	运营VP(她在意)
5-7min	不止是DAU——用户体验也在降	「核心行为完成率从68%→51%，NPS降了13分」	HEART(概念6)	产品总监(她在意)
7-8min	下月预测——区间而非点	「下月DAU 95% CI [115,145]万，留存率是最关键驱动因素」	蒙特卡洛(概念7)	CEO(要数字)
8-9min	建议——换种方式投	「暂停次留<20%渠道(C级)，省下的预算投产品优化+高质渠道」	博弈论(概念1)	投放总监+CEO
9-10min	行动+指标	「目标：6个月Retained回到70万。下周启动留存专项，每月汇报PDS+HEART联合健康度」	看板SQL(概念5)	全员(共识)

关键技巧：第一句话给结论（CEO 喜欢的），数据给区间的（展现专业度），建议给替代方案的（投放总监不会觉得你在砍他预算而是重分配）。

🎯 综合题 2：沙盘变体——如果闪念是一家电商 App

场景：闪念从短视频产品转型为社交电商（短视频种草 + 直播带货）。用户行为模式完全改变——不再是「每天刷刷视频」，而是「有购物需求时才打开」。

任务：

1. 电商 App 的健康 Retained 基准是多少？为什么比短视频低？
2. 电商 App 的「虚胖」长什么样？（提示：大促期间的 DAU 暴涨是虚胖吗？）
3. 虚胖自动检测器的 5 条规则在电商场景下需要怎么调整？

参考答案

1. 电商健康基准：Retained > 30%（vs 短视频的 55%）。电商是「非每日刚需」——用户不需要每天买东西。低 Retained 不代表产品不好，而是品类特征。应该用「月度留存率」替代「日留存率」作为核心 KPI。

2. 电商虚胖的典型特征：大促期间（双11/618）DAU 暴涨 200%，但 80% 的增量是「羊毛党 + 一次性用户」。大促结束 → DAU 跌回甚至低于大促前（因为用户囤货后接下来几周不买了）。虚胖信号不是「DAU 涨了」而是「大促后第 2 周 DAU 低于大促前第 2 周」——说明大促没有转化为长期用户。

3. 检测器规则调整：

- 规则 1（New 占比阈值）：从 30% 调整为按「是否大促期间」判断——非大促期间 New > 25% 即为异常
- 规则 4（Retained 趋势）：从看日 Retained 改为看周 Retained 或月 Retained
- 新增规则 6（大促透支检测）：大促后 14 天 DAU < 大促前 14 天 DAU × 0.95 → 大促在透支而非增长

🎯 综合题 3：边界判断——什么场景下 PDS 不适用或需要修正？

判断以下 3 个场景：PDS 框架是否适用？如果适用，需要做什么修正？

场景 A：「闪念」刚上线 3 天，只有 2000 个注册用户。能用 PDS 判断增长健康度吗？

场景 B：闪念是一个 B2B SaaS 工具（项目管理软件），客单价 5 万/年。DAU 的定义是「至少打开一次项目页面」。能用 PDS 吗?

场景 C：闪念推出「小程序版」——用户在微信内使用，不需要下载 App。小程序的数据在微信后台，只能拿到「打开次数」和「新增/留存」，拿不到具体的用户级别的行为数据。能用 PDS 吗?

参考答案

场景 A：❌ 不适用。3 天数据、2000 用户的 PDS 占比完全不可靠（Bootstrap CI 会极宽）。冷启动阶段应该用 HEART 框架关注「Adoption（采纳率）」和「Task Success（核心行为完成率）」——验证产品是否有 PMF（Product-Market Fit），而不是分析 DAU 结构。

场景 B：⚠️ 部分适用，但需要重新定义「活跃」。B2B SaaS 的关键不是「打开项目页面」这种轻度行为（约等于 B2C 的 push 点击），而是「完成了核心工作」——创建了任务、评论了文档、完成了审批。如果 DAU 的定义太松 → PDS 中的 Retained 占比会被高估（很多「开了但不干活」的人被算成了 Retained）。修正：用「核心行为 DAU」（完成至少 1 个核心动作的 DAU）替代「打开即 DAU」。

场景 C：⚠️ 数据受限，只能做「简化版 PDS」。微信后台给的新增/留存是基于微信的设备 ID 的——换手机 = 新用户。而且无法区分 Resurrected（因为拿不到用户的全部历史活跃记录）。修正：在小程序后台拿到 DAU 和 New，但 Retained 和 Resurrected 可能需要用服务端日志（如果有）来补全。如果数据确实不全→坦诚标注 PDS 报告的「数据质量声明」。

核心教训：PDS 是一个框架——框架的输入是干净的、用户级别的活跃数据。如果数据不干净、定义不恰当、样本量太小 → 框架再对、结论也是错的。在应用任何框架之前，先问：「数据本身支持这个框架吗？」

🎯 综合题 4：落地——为闪念设计一份完整的 PDS 监控体系

场景：运营 VP 王姐看完你的分析后说：「你分析的很好。但我不希望你每天手动分析——下个月之前，你给我设计一套自动化的 PDS 监控体系。要求：每天早上 9 点自动更新、异常自动告警到飞书群、看板可自助查询。」

任务：设计这套监控体系——包括数据流、计算频率、告警规则、看板布局。这是你从「分析师」升级为「数据产品经理」的关键练习。

参考答案（监控体系设计）

数据流：

```
客户端埋点 → Kafka → Flink实时ETL → Hive ODS层(user_active_log)
  → 每日T+1凌晨2点：Spark SQL计算DWS层(dws_dau_pds_daily)
  → 每日T+1凌晨3点：Airflow DAG运行：
      ① PDS计算（概念5 SQL）
      ② Bootstrap CI计算（概念2）
      ③ 虚胖检测器运行（概念3 Python类）
      ④ 蒙特卡洛预测更新（概念7）
      ⑤ 告警判断 + 飞书推送
  → 每日T+1早上8点：看板刷新（Metabase/Superset/Grafana）
```

看板布局（一页纸）：

- 左上：DAU趋势图（过去12周）+ PDS三元堆叠面积图
- 右上：健康评级大号显示（🟢🟡🔴）+ 告警列表
- 左下：Retained绝对值12周趋势线（最核心的一张图）
- 右下：Retained N日留存矩阵热力图
- 底部：渠道分级表（A/B/C级 + 次留率 + 预算分配）+ 蒙特卡洛预测区间

告警规则（飞书推送）：

- 🔴 红：Retained连续5周↓，或周环比降>10%，或虚胖指数>5 → 推送运营VP+产品总监+数据负责人
- 🟡 黄：Retained连续3周↓，或New占比>35%，或次留<20%渠道的新增占比>30% → 推送数据团队+对应渠道负责人
- 🟣 观察：Resurrected连续3周↑ → 仅数据团队内部记录

第五部分：知识网络图

概念关系全景表

概念	关系类型	与PDS的关联	深度类型	来源
1. PDS博弈论	[防护]	防止PDS指标被「做」而非「做好」	博弈+案例	永久空白区
2. Bootstrap CI	[前置]	PDS贡献度分析需要「区间」而非「点」	数学+代码	永久空白区
3. 虚胖检测器	[防护]	PDS诊断能力的自动化封装	代码	永久空白区

概念	关系类型	与PDS的关联	深度类型	来源
4. 数据生产陷阱	[并行]	PDS数据的质量是分析正确性的前提	工程+代码	工程实践
5. 监控看板SQL	[落地]	PDS从分析技能→数据产品	工程+代码	工程实践
6. HEART框架	[替代]	PDS看行为结构，HEART看体验质量	案例	永久空白区
7. 蒙特卡洛预测	[核心深化]	PDS预测从点估计→区间估计	数学+代码	数学深化

协同使用指南：

- **日常监控**：概念 3（检测器）+ 概念 5（看板 SQL）→ 自动化盯盘
- **深度诊断**：概念 2（Bootstrap）+ 概念 4（数据质量）+ 概念 6（HEART）→ 多维度交叉验证
- **向上汇报**：概念 1（博弈论）+ 概念 7（蒙特卡洛预测）→ 说服管理层+管理预期

三版本覆盖对比

知识点	主课程覆盖	已有拓展覆盖	本拓展覆盖
PDS公式+增长模式判断	✅ 完整	—	—
PDS虚胖识别+贡献度	✅ 完整	—	—
PDS周报+场景应用	✅ 完整	—	—
Bootstrap置信区间	❌ 未覆盖	❌ 未覆盖	✅ 概念2
指标博弈(Goodhart's Law)	❌ 未覆盖	❌ 未覆盖	✅ 概念1
虚胖DAU自动检测器	❌ 未覆盖	❌ 未覆盖	✅ 概念3
数据生产陷阱(埋点/时区)	❌ 未覆盖	❌ 未覆盖	✅ 概念4
生产级PDS SQL	❌ 未覆盖	❌ 未覆盖	✅ 概念5
HEART vs PDS	❌ 未覆盖	❌ 未覆盖	✅ 概念6
蒙特卡洛预测区间	❌ 未覆盖	❌ 未覆盖	✅ 概念7
PDS预测(三情景)	✅ 完整	—	概念7在此基础上深化

覆盖盲区总结：主课覆盖了 PDS 的「分析层」（怎么拆、怎么看、怎么判断），本拓展覆盖了 PDS 的「基础设施层」（数据质量→计算方法→自动化→预测区间→博弈防护→盲区弥补）。两层叠加 → 完整的 PDS 知识体系。

延伸阅读

主题	推荐材料	难度	预计时间
Bootstrap深入	Efron & Tibshirani《An Introduction to the Bootstrap》	中	2h
Goodhart's Law	Strathern(1997)「Improving Ratings」论文	易	30min
HEART框架原文	Rodden et al.(2010) Google CHI论文	中	1h
蒙特卡洛方法	Rubinstein & Kroese《Simulation and the Monte Carlo Method》	难	4h
数据质量工程	Great Expectations官方文档 + Pandera文档	中	2h
博弈论与机制设计	秦曾昌译《微观动机与宏观行为》(Schelling)	中	3h
预测与校准	Tetlock《Superforecasting》	易	4h
后续课程	W3 ISS系统化分析(W3D2)——PDS发现问题后如何系统诊断	—	—
后续课程	W7-W8 AB实验——验证「改进产品后Retained是否提升」	—	—

跨模块连接

连接方向	关联内容	如何协同
← W1D2 指标设计	Goodhart's Law + 博弈论	设计「博弈免疫」的PDS指标体系
← W1D6 异动分析	Bootstrap CI + 数据质量检查	异动分析前先确认数据干净+变化显著
← W2D2 同期群	Retained的N日留存矩阵	Bootstrap可以用于同期群留存率的CI估算
→ W3D2 ISS框架	PDS发现问题 → ISS系统诊断	本拓展教你「怎么发现和量化问题」→W3教你「怎么系统解决问题」
→ W3D3 补贴ROI	博弈论延伸	补贴的ROI分析也要考虑「补贴被博弈」的问题
→ W7-W8 AB实验	改进产品的留存效果验证	蒙特卡洛模拟可用于AB实验的MDE敏感性分析

连接方向	关联内容	如何协同
→ W9 因果推断	DID/PSM的Bootstrap标准误	Bootstrap也可以用于因果推断模型的标准误估计
→ W11 综合项目	完整的监控体系+预测模型	本拓展的检测器+看板SQL+预测是W11综合项目的核心技术组件

第六部分：元认知自评

学习循环自评表

阶段	自检问题	你的回答	如果答不上来...
吸收	我能不翻笔记说出7个概念的名称和关系类型吗?		回看「概念关系全景表」
理解	我能向非技术同事用一句话解释每个概念吗?		回看每个概念的「①一句白话」
应用	我能用闪念的数据独立跑一遍 Bootstrap + 检测器 + 蒙特卡洛吗?		重新运行代码cell并修改参数
迁移	如果明天换到电商/游戏/工具产品，我能调整 PDS 基准和检测器阈值吗?		回顾综合题2+3
创造	我能为另一个产品（如你当前工作的产品）设计完整的 PDS 监控体系吗?		回顾综合题4的参考答案作为模板

多维度自评量表

对以下 6 个维度自我评分（0=完全不会, 1=能做但有卡点, 2=熟练）：

维度	能力描述	评分
PDS诊断力	能在30秒内从PDS数据判断增长健康度	/2
统计思维	能用Bootstrap/蒙特卡洛量化不确定性	/2
工程化能力	能写出生产级PDS SQL并设计自动化流程	/2
博弈思维	能预见指标被博弈的方式并设计防护	/2

维度	能力描述	评分
数据质量意识	能在分析之前先检查数据是否「干净」	/2
综合沟通力	能用数据+故事向上说服管理层	/2

总分：___/12

- 10-12 分：PDS 知识体系扎实，可以进入 W3
- 7-9 分：建议花 30 分钟回看薄弱维度的概念
- <7 分：建议重新做综合题 1-4，重点补齐短板

薄弱点精准强化方案

根据你的自评结果，对照以下方案：

- **PDS 诊断力 < 2**：回主课重做 cell-6 和 cell-8 的练习
- **统计思维 < 2**：重跑概念 2 和概念 7 的代码，把参数（样本量、分布类型、置信水平）各改 3 次看结果怎么变
- **工程化能力 < 2**：把概念 5 的 SQL 手写一遍（不看参考答案），然后在你自己环境的数据库中建模拟表跑通
- **博弈思维 < 2**：拿 3 个你日常工作/生活中遇到的 KPI，用「博弈免疫三条检验」逐个检验
- **数据质量意识 < 2**：找一份你们团队的历史数据，用概念 4 的 5 项检查清单逐项检查
- **综合沟通力 < 2**：用概念 6 的 HEART+PDS 联合矩阵，写一份你们产品的一页纸健康度报告

沙盘复述测试

测试方法：不翻前面的任何笔记，口头回答以下 5 个问题（建议录音然后自己听）：

1. 闪念科技是什么公司？DAU 多少？当前面临什么挑战？
2. 你的角色是谁？你要向谁汇报？他们要什么？
3. 你用哪 7 个概念解决了闪念的什么问题？（每个概念一句话）
4. 如果闪念的 CEO 明天让你做下月 DAU 预测——你会怎么回答？（包含什么数字？以什么形式呈现？）
5. 如果闪念的产品总监问你「DAU 在涨但用户真的快乐吗」——你用什么框架回答？

合格标准：每个问题在 30 秒内给出清晰、有结构的回答。如果磕磕绊绊 → 说明理解和记忆还不够深 → 回看对应概念。

循环毕业检查清单

完成以下所有项才能说「我学完了这个拓展」：

- [ ] 自诊 5 题全部答对（或有错题已对照补救路径复习）
- [ ] 概念 2 的 Bootstrap 代码能在自己环境跑通并理解输出
- [ ] 概念 3 的虚胖检测器 `VanityDAUDetector` 类能独立实现（不看参考答案）
- [ ] 概念 5 的 PDS SQL 能手写（不看参考答案）
- [ ] 概念 7 的蒙特卡洛模拟能理解并修改参数重跑
- [ ] 综合沙盘 4 题全部完成（口头或书面）
- [ ] 元认知自评得分 ≥ 8 分
- [ ] 沙盘复述 5 题能在 30 秒/题内清晰回答
- [ ] 能向一个「完全不懂 PDS」的同事解释：DAU 为什么会撒谎？怎么发现它在撒谎？

全部打勾 → 🎉 恭喜！PDS 模块毕业。你的 DAU 分析能力已经从「会看数字」升级到「会建体系」。

03analyst_thinking · 拓展_W2D6_PDS深度网络 · 2026-07-01